

bok25

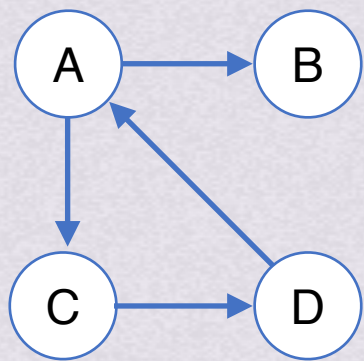
基
礎
班

、**考研数据結構**



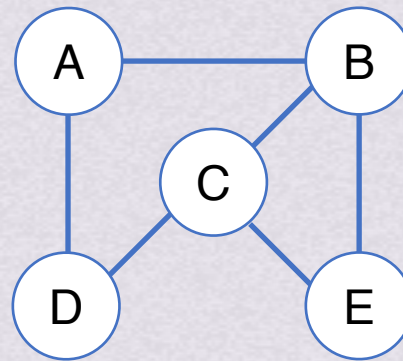
图的存储结构邻接矩阵

邻接矩阵：用来表示顶点之间相邻关系的矩阵



有向图 G_1

$$G_{1,arcs} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$



无向图 G_2

$$G_{2,arcs} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

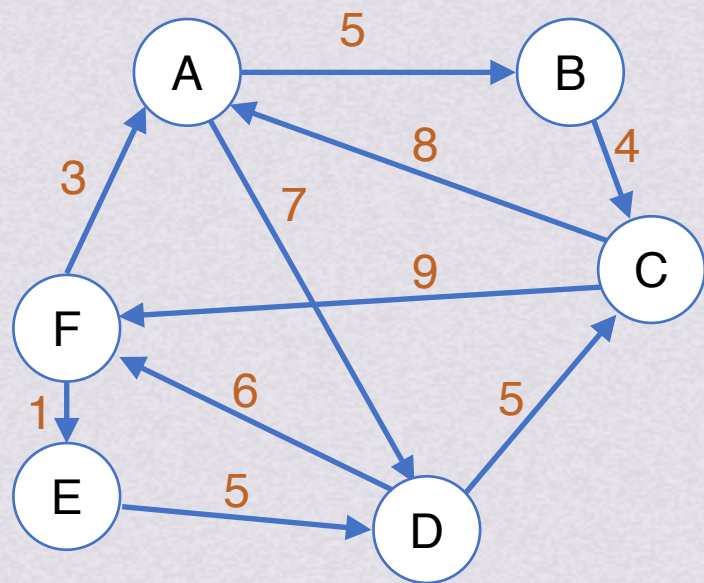
$$\text{对于图: } A[i][j] = \begin{cases} 1 & \langle v_i, j_i \rangle \text{ 或 } (v_j, j_i) \in E \\ 0 & \text{其他} \end{cases}$$

$$\text{对于网: } A[i][j] = \begin{cases} w_{ij} & \langle v_i, j_i \rangle \text{ 或 } (v_j, j_i) \in E \\ \infty & \text{其他} \end{cases}$$



图的存储结构邻接矩阵

邻接矩阵：用来表示顶点之间相邻关系的矩阵



有向网 G_3

$$G_{3,arcs} = \begin{bmatrix} \infty & 5 & \infty & 7 & \infty & \infty \\ \infty & \infty & 4 & \infty & \infty & \infty \\ 8 & \infty & \infty & \infty & \infty & 9 \\ \infty & \infty & 5 & \infty & \infty & 6 \\ \infty & \infty & \infty & 5 & \infty & \infty \\ 3 & \infty & \infty & \infty & 1 & \infty \end{bmatrix}$$

$$\text{对于图: } A[i][j] = \begin{cases} 1 & \langle v_i, v_j \rangle \text{ 或 } (v_j, v_i) \in E \\ 0 & \text{其他} \end{cases}$$

$$\text{对于网: } A[i][j] = \begin{cases} w_{ij} & \langle v_i, v_j \rangle \text{ 或 } (v_j, v_i) \in E \\ \infty & \text{其他} \end{cases}$$



图的存储结构

邻接矩阵表示法的结构体

```
//----- 图的邻接矩阵存储表示 -----  
#define MaxInt 32767 //表示极大值, 即 $\infty$   
#define MVNum 100 //最大顶点数  
typedef struct{  
    VerTexType vexs[MVNum]; //顶点表  
    ArcType arcs[MVNum][MVNum]; //邻接矩阵  
    int vexnum, arcnum; //图的当前点数和边数  
}AMGraph;
```



图的存储结构

邻接矩阵表示法的结构体

```
//----- 图的邻接矩阵存储表示 -----  
#define MaxInt 32767 //表示极大值, 即 $\infty$   
#define MVNum 100 //最大顶点数  
typedef struct{  
    VerTexType vexs[MVNum]; //顶点表  
    ArcType arcs[MVNum][MVNum]; //邻接矩阵  
    int vexnum, arcnum; //图的当前点数和边数  
}AMGraph;
```




图的存储结构

邻接矩阵表示法的结构体

顶点
(同vertices)

```
//----- 图的邻接矩阵存储表示 -----  
#define MaxInt 32767 //表示极大值, 即 $\infty$   
#define MVNum 100 //最大顶点数  
typedef struct{  
    VerTexType vexs[MVNum]; //顶点表  
    ArcType arcs[MVNum][MVNum]; //邻接矩阵  
    int vexnum, arcnum; //图的当前点数和边数  
}AMGraph;
```



图的存储结构

邻接矩阵表示法的结构体

```
//----- 图的邻接矩阵存储表示 -----  
#define MaxInt 32767 //表示极大值, 即 $\infty$   
#define MVNum 100 //最大顶点数  
typedef struct { 顶点的数据类型  
    VerTexType vexs[MVNum]; //顶点表  
    ArcType arcs[MVNum][MVNum]; //邻接矩阵  
    int vexnum, arcnum; //图的当前点数和边数  
}AMGraph;
```




图的存储结构

邻接矩阵表示法的结构体

```
//----- 图的邻接矩阵存储表示 -----  
#define MaxInt 32767 //表示极大值, 即 $\infty$   
#define MVNum 100 //最大顶点数  
typedef struct{  
    VerTexType vexs[MVNum]; //顶点表  
    ArcType arcs[MVNum][MVNum]; //邻接矩阵  
    int vexnum, arcnum; //图的当前点数和边数  
}AMGraph; //弧/边
```




图的存储结构

邻接矩阵表示法的结构体

```
//----- 图的邻接矩阵存储表示 -----  
#define MaxInt 32767 //表示极大值, 即 $\infty$   
#define MVNum 100 //最大顶点数  
typedef struct{  
    VerTexType vexs[MVNum]; //顶点表  
    ArcType arcs[MVNum][MVNum]; //邻接矩阵  
    int vexnum, arcnum; //图的当前点数和边数  
}AMGraph;
```

弧/边上的权值的数据类型



图的存储结构

邻接矩阵表示法的结构体

```
//----- 图的邻接矩阵存储表示 -----  
#define MaxInt 32767 //表示极大值, 即 $\infty$   
#define MVNum 100 //最大顶点数  
typedef struct{  
    VerTexType vexs[MVNum]; //顶点表  
    ArcType arcs[MVNum][MVNum]; //邻接矩阵  
    int vexnum, arcnum; //图的当前点数和边数  
}AMGraph;
```

vertex的缩写
即顶点



图的存储结构

邻接矩阵表示法

优缺点总结

优点

- 便于判断两个顶点之间是否有边
- 便于计算各个顶点的度：

对于无向图：

顶点 v_i 的度=

邻接矩阵第 i 行元素之和=

邻接矩阵第 i 列元素之和

对于有向图：

- 顶点 v_i 的出度=第 i 行元素之和
- 顶点 v_i 的入度=第 i 列元素之和

缺点

- 不便于增加和删除节点。
- 不便于统计边的数目，需要遍历邻接矩阵才能统计，时间复杂度为 $O(n^2)$
- 空间复杂度高：只适合稠密图，稀疏图会造成空间的浪费
 - 有向图： n 个顶点的图需要 n^2 个单元存储边
 - 无向图：邻接矩阵是对称的，因此可以压缩存储，仅存上三角/下三角，只需 $\frac{n(n-1)}{2}$ 个空间。但空间复杂度仍为 $O(n^2)$



图的存储结构

邻接表表示法

邻接表

表头节点表

- 由所有表头节点以顺序结构的形式存储，以便可以随机访问任一顶点的边链表。
- 表头节点包括数据域data和链域firstarc
- 数据域用于存储顶点 v_i 的数据，链域firstarc用于指向链表中第一个节点（也就是与顶点 v_i 邻接的第一个邻接点）

边表

- 由记录顶点间关系的 n 个边链表组成
- 边链表中的边节点包括邻接点域adjvex、数据域info和链域nextarc
- 邻接点域adjvex指示与顶点 v_i 邻接的点在图中的位置，数据域info存储和边相关的数据，链域firstarc指示与顶点 v_i 邻接的下一条边的节点



图的存储结构邻接表表示法

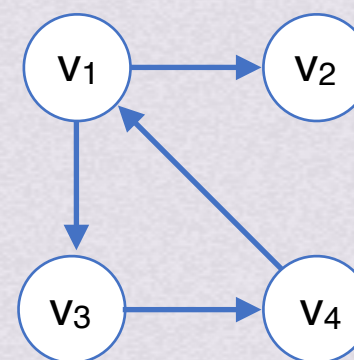
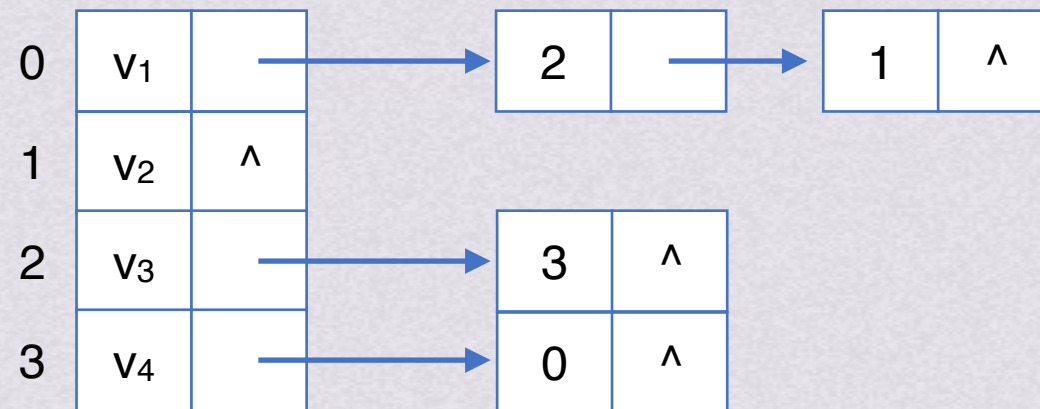
表头节点表

- 由所有表头节点以顺序结构的形式存储，以便可以随机访问任一顶点的边链表。
- 表头节点包括数据域data和链域firstarc
- 数据域用于存储顶点 v_i 的数据，链域firstarc用于指向链表中第一个节点（也就是与顶点 v_i 邻接的第一个邻接点）

邻接表

表头节点表

边表



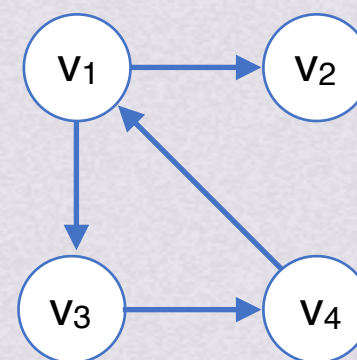
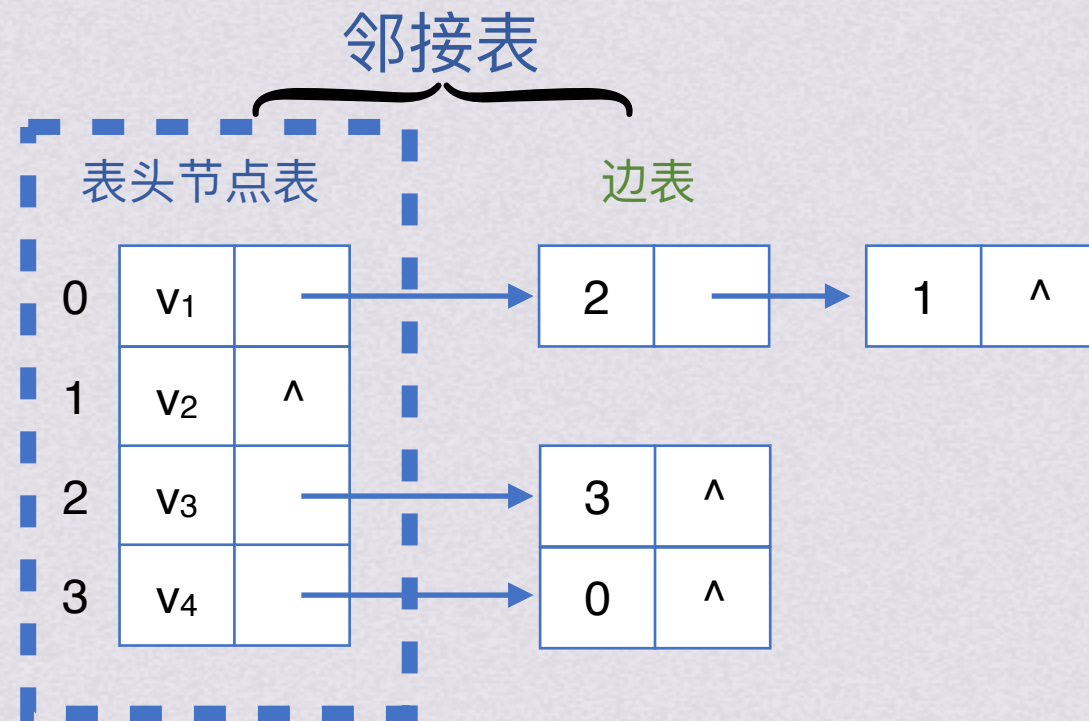
有向图 G_1



图的存储结构邻接表表示法

表头节点表

- 由所有表头节点以顺序结构的形式存储，以便可以随机访问任一顶点的边链表。
- 表头节点包括数据域data和链域firstarc
- 数据域用于存储顶点 v_i 的数据，链域firstarc用于指向链表中第一个节点（也就是与顶点 v_i 邻接的第一个邻接点）



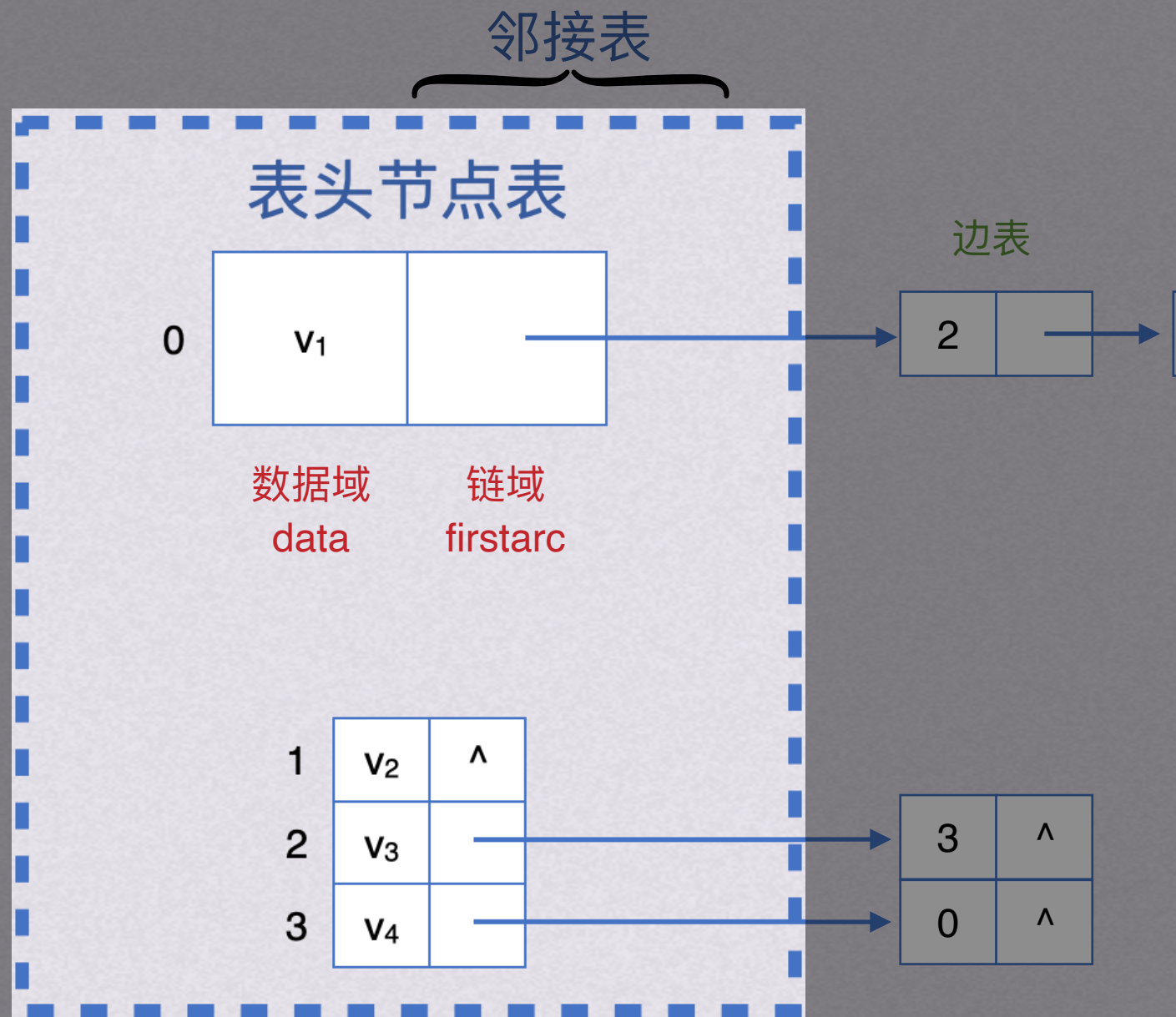
有向图 G_1



图的存储结构邻接表表示法

表头节点表

- 由所有表头节点以顺序结构的形式存储，以便可以随机访问任一顶点的边链表。
- 表头节点包括数据域data和链域firstarc
- 数据域用于存储顶点 v_i 的数据，链域firstarc用于指向链表中第一个节点（也就是与顶点 v_i 邻接的第一个邻接点）

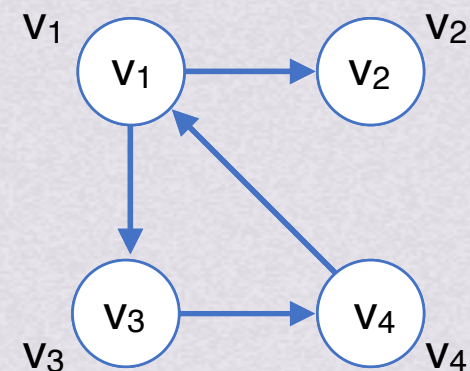
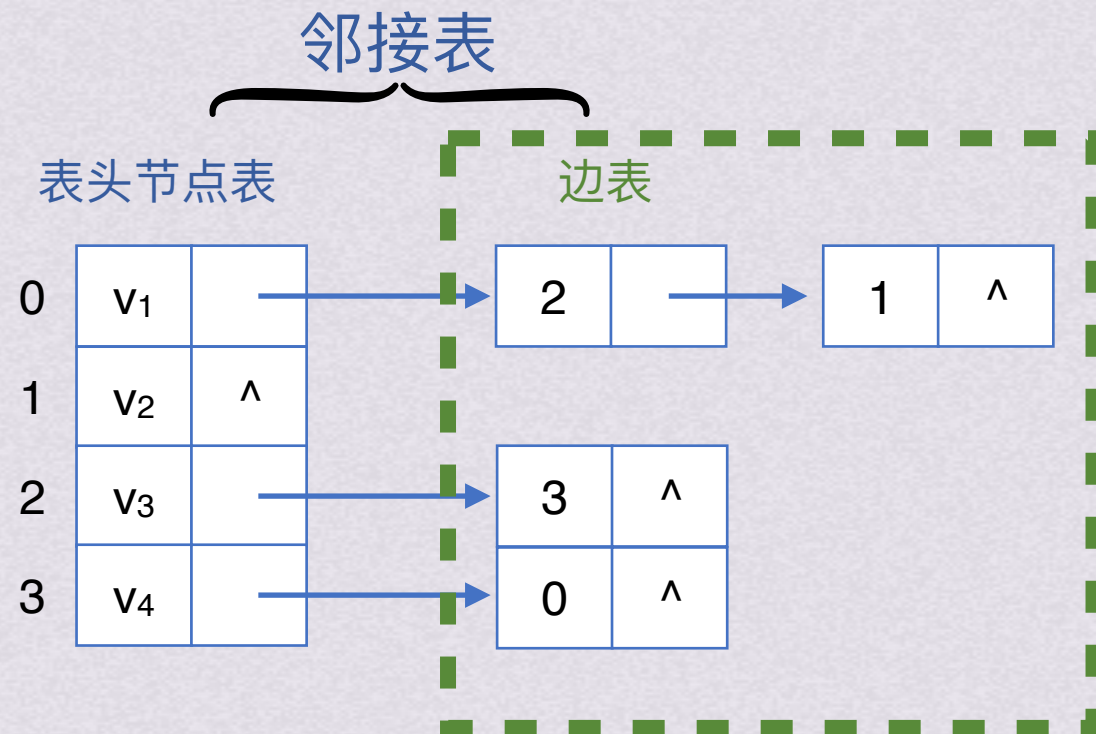




图的存储结构邻接表表示法

边表

- 由记录顶点间关系的n个边链表组成
- 边链表中的边节点包括邻接点域adjvex、数据域info和链域nextarc
- 邻接点域adjvex指示与顶点 v_i 邻接的点在图中的位置，数据域info存储和边相关的数据，链域firstarc指示与顶点 v_i 邻接的下一条边的节点



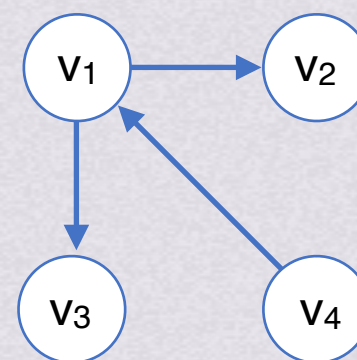
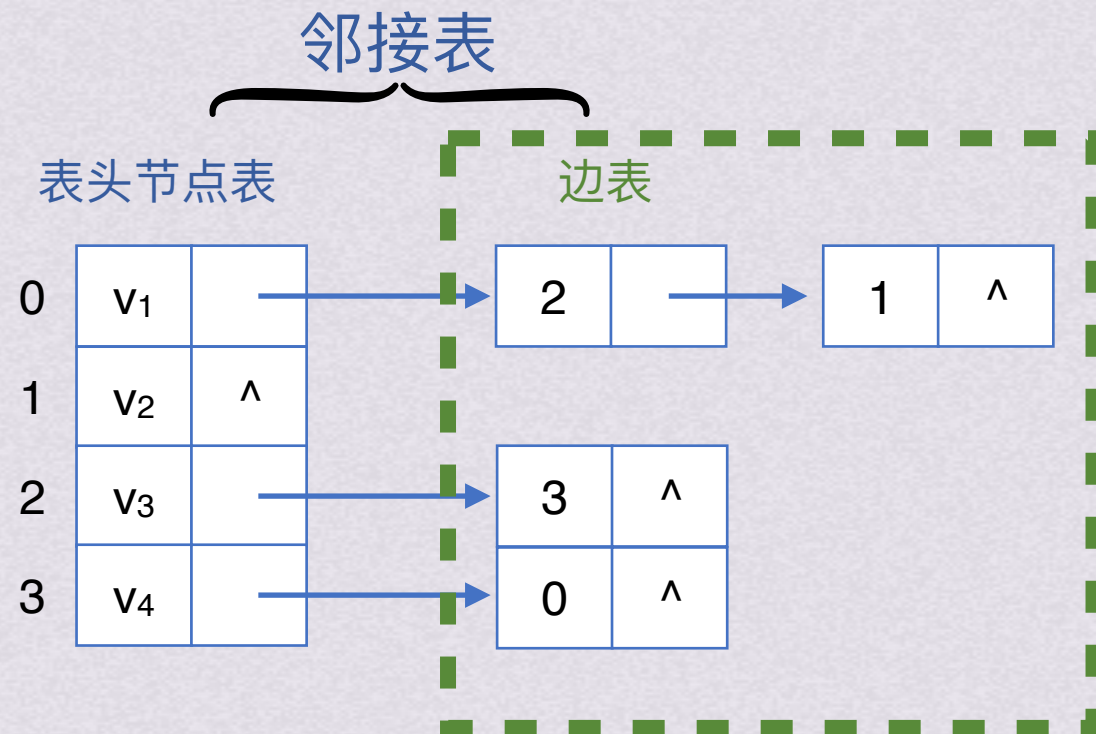
有向图 G_1



图的存储结构邻接表表示法

边表

- 由记录顶点间关系的n个边链表组成
- 边链表中的边节点包括邻接点域adjvex、数据域info和链域nextarc
- 邻接点域adjvex指示与顶点 v_i 邻接的点在图中的位置，数据域info存储和边相关的数据，链域firstarc指示与顶点 v_i 邻接的下一条边的节点

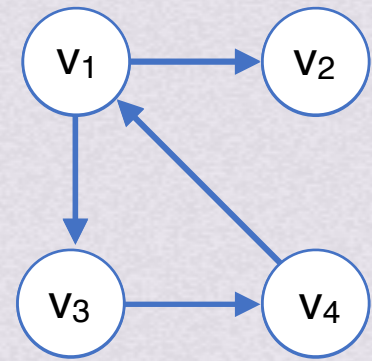


有向图 G_1

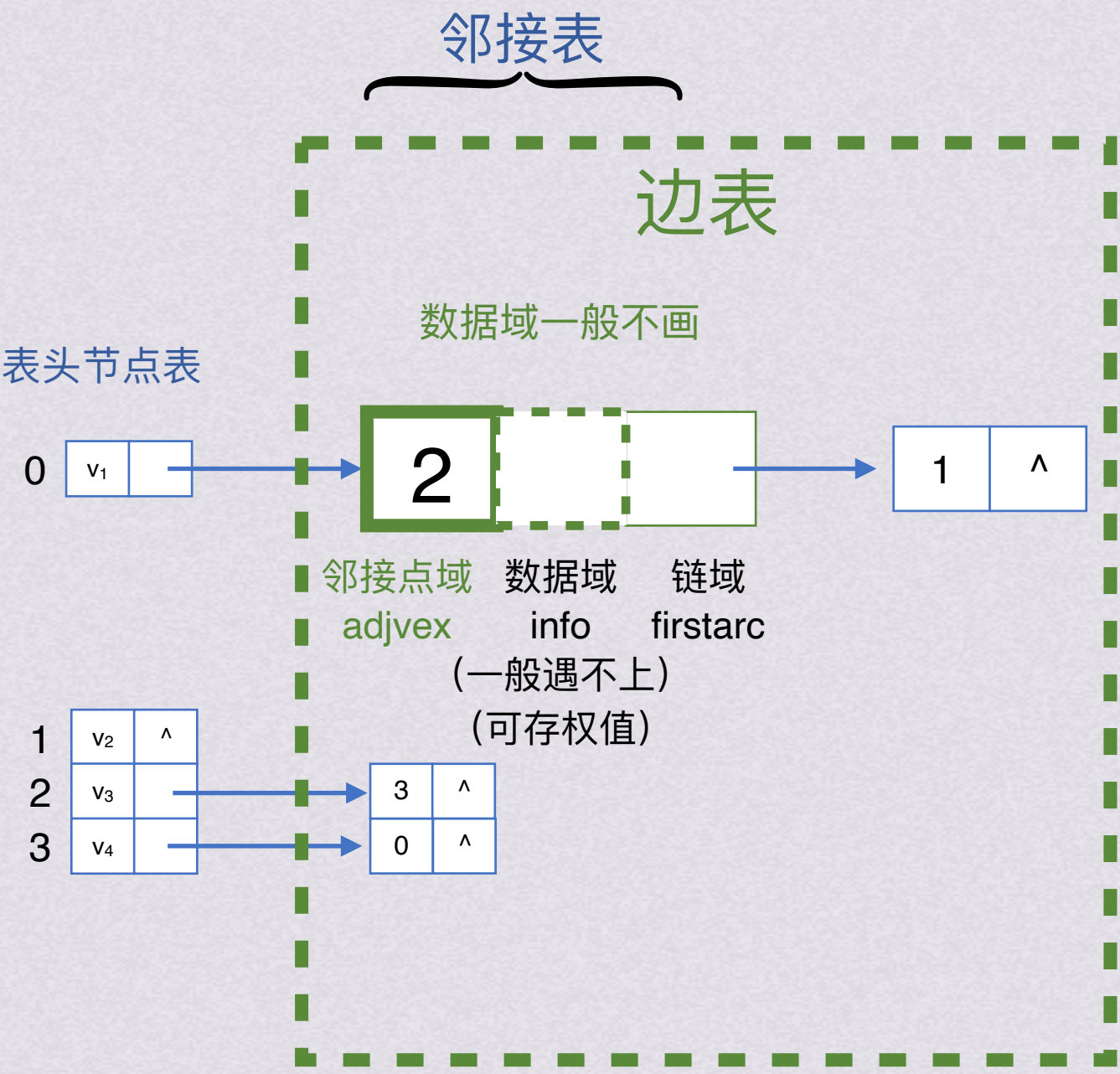
图的存储结构邻接表表示法

边表

- 由记录顶点间关系的n个边链表组成
- 边链表中的边节点包括邻接点域adjvex、数据域info和链域nextarc
- 邻接点域adjvex指示与顶点vi邻接的点在图中的位置，数据域info存储和边相关的数据，链域firstarc指示与顶点vi邻接的下一条边的节点

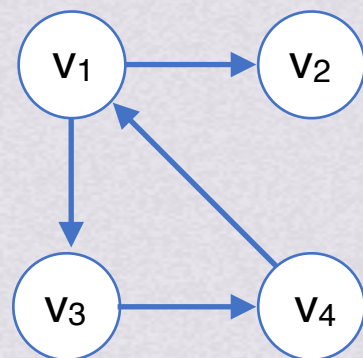
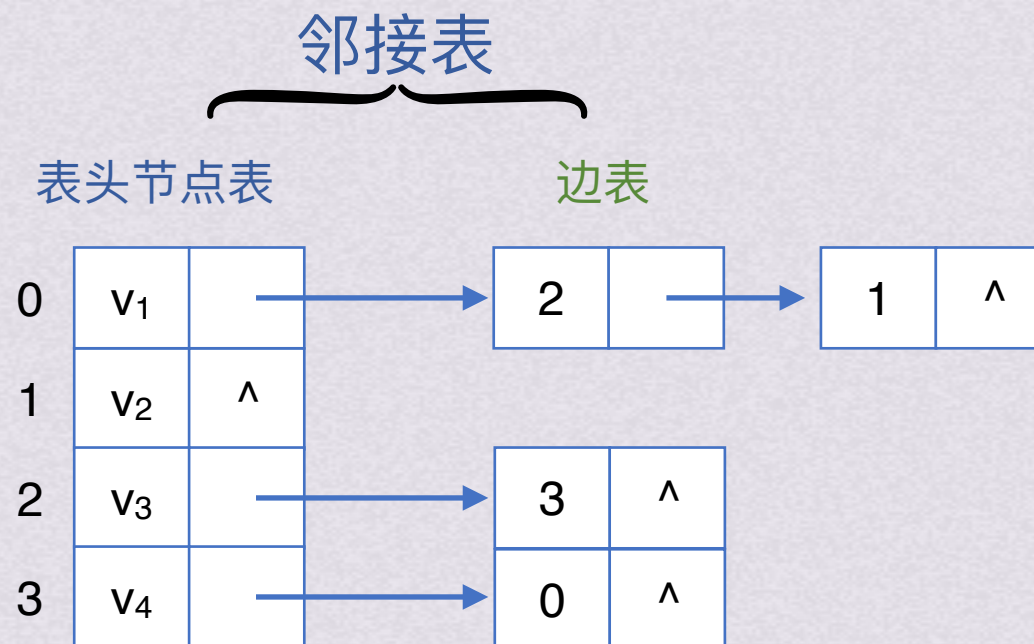


有向图G₁





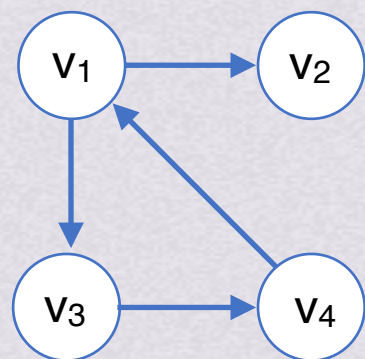
图的存储结构邻接表表示法

有向图 G_1 

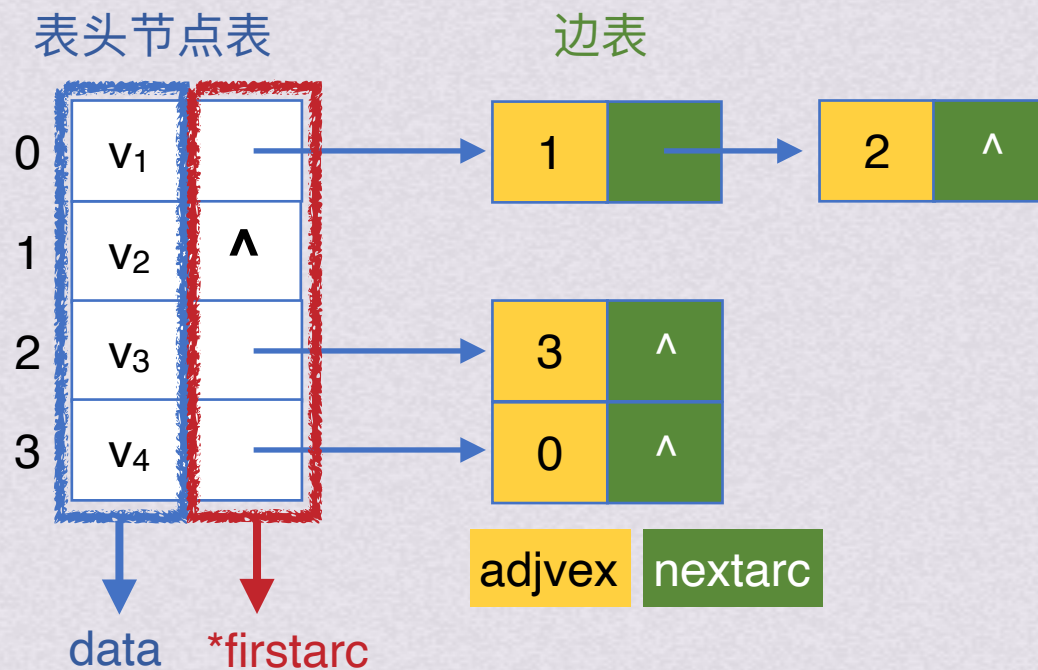
试着将图对应的邻接表自己画出来！



图的存储结构邻接表表示法

有向图 G_1

G_1 对应邻接表



表头节点

data	firstarc
------	----------

VNode

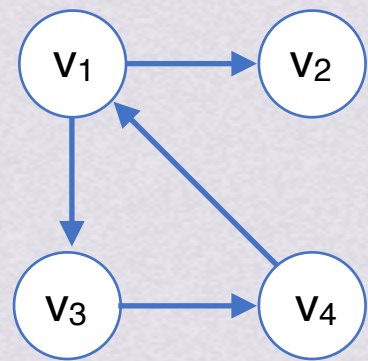
边节点

adjvex	info	nextarc
--------	------	---------

ArcNode

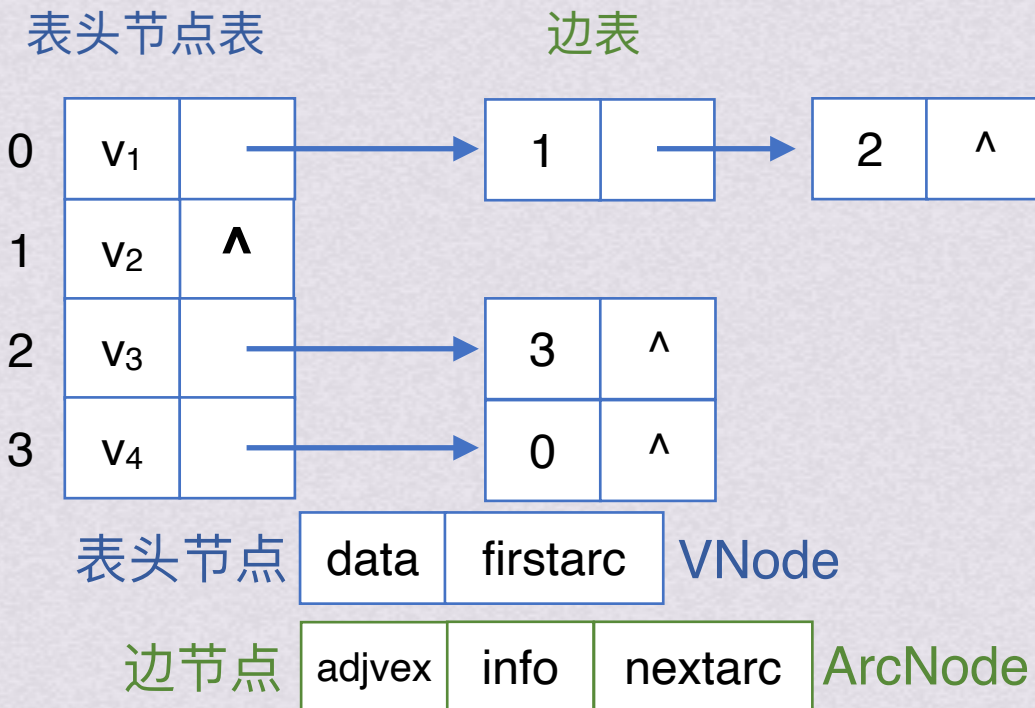
图的存储结构邻接表表示法

```
//- - - - -图的邻接表存储表示- - - - -
#define MVNum 100           //最大顶点数
typedef struct ArcNode {    //边节点
    int adjvex;             //该边所指向的顶点的位置
    struct ArcNode * nextarc; //指向下一条边的指针
    OtherInfo info;         //和边相关的信息
}ArcNode;
typedef struct VNode        //顶点信息
{
    VerTexType data;
    ArcNode *firstarc;      //指向第一条依附该顶点的边的指针
}VNode, AdjList[MVNum];    //AdjList 表示邻接表类型
typedef struct {            //邻接表
    AdjList vertices;
    int vexnum, arcnum;     //图的当前顶点数和边数
}ALGraph;
```



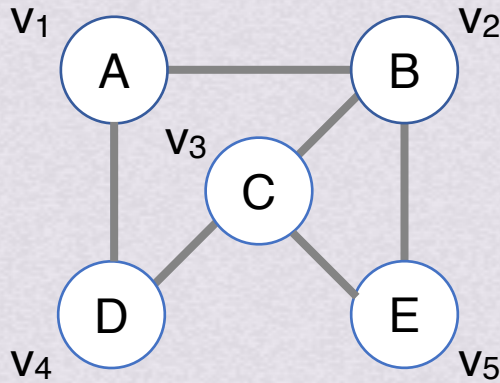
有向图G₁

G₁对应邻接表

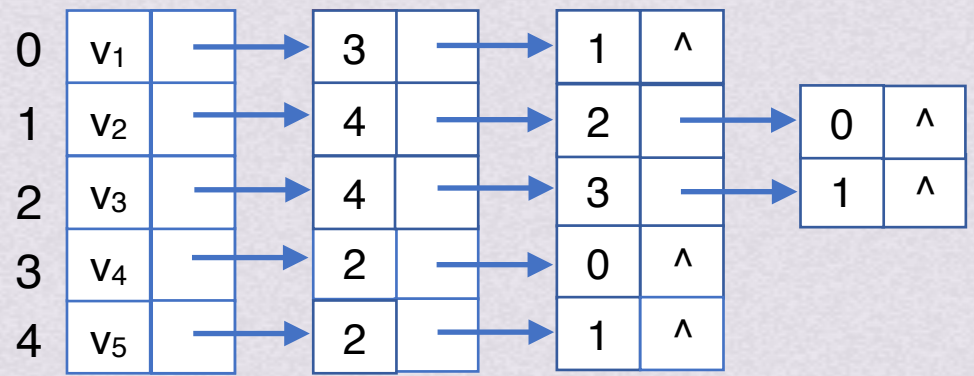
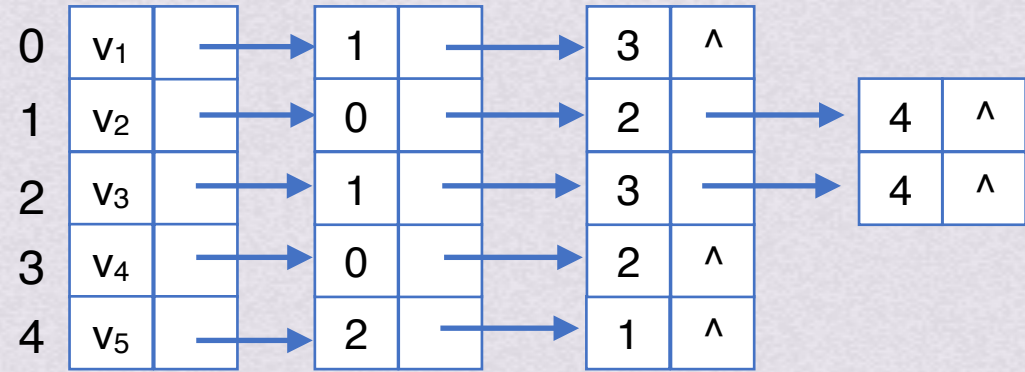


图的存储结构

邻接表表示法 邻接表生成无向图



目标无向图G₂





图的存储结构

邻接表表示法

优缺点总结

优点

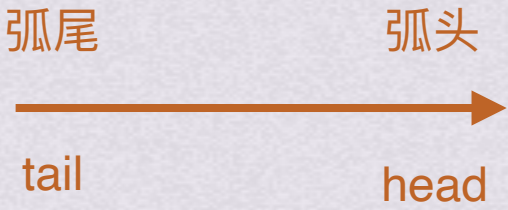
- 便于增加和删除节点
- 便于统计边的数目，按顶点表顺序查找所有边表即可得到边的数目，时间复杂度为 $O(n+e)$
- 空间效率高：空间复杂度只需 $O(n+e)$ ，适合稀疏图
 - n 个顶点、 e 条边的无向图： n 个顶点表节点和 $2e$ 个边表节点。
 - n 个顶点、 e 条边的有向图： n 个顶点表节点和 e 个边表节点

缺点

- 不便于判断顶点之间是否有边。如果要判定 v_i 和 v_j 中是否有边，需要查找第 i 个边表，最坏情况下时间复杂度为 $O(n)$
- 不便于计算有向图各个顶点的度
 - 无向图：顶点 v_i 的度是第 i 个边表中的节点个数
 - 有向图：顶点 v_i 的出度是第 i 个边表中的节点个数，但求顶点 v_i 的入度需要遍历各顶点的边表

图的存储结构十字链表

顶点节点

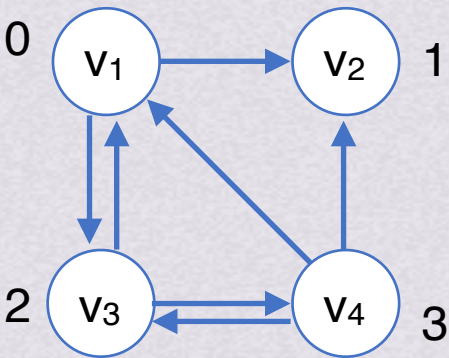


节点编号

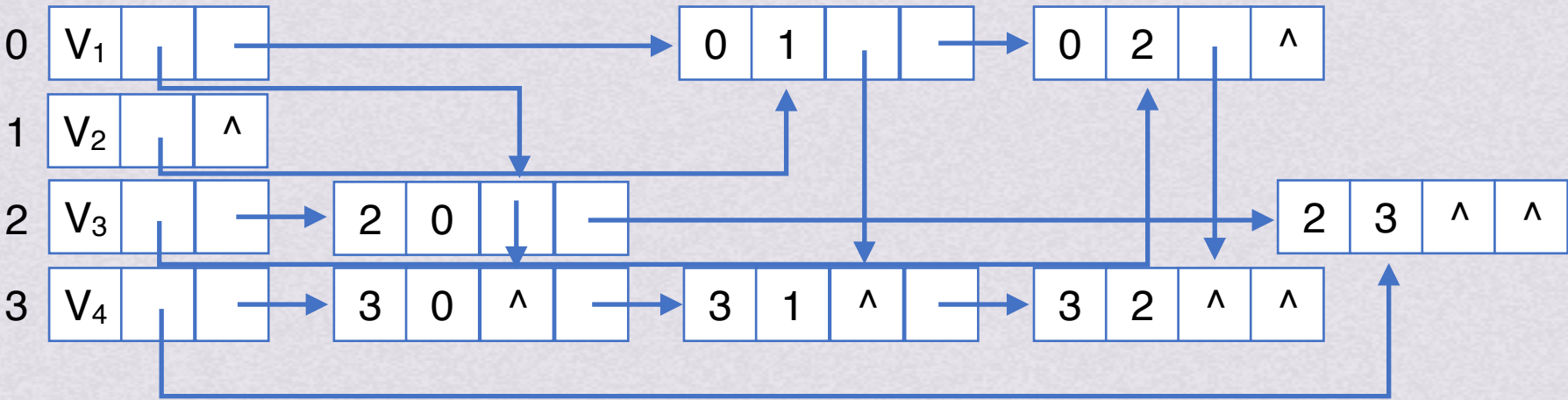
data	firstin	firstout
------	---------	----------

弧节点

tailvex	headvex	hlink	tlink	(info)
---------	---------	-------	-------	--------



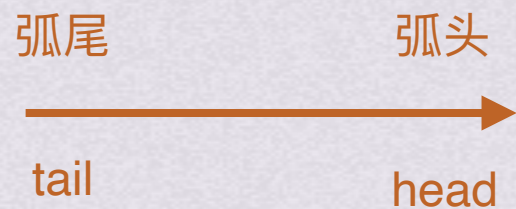
有向图G₃



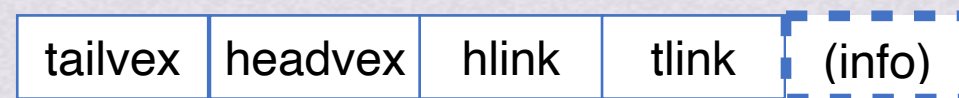


图的存储结构十字链表

顶点节点

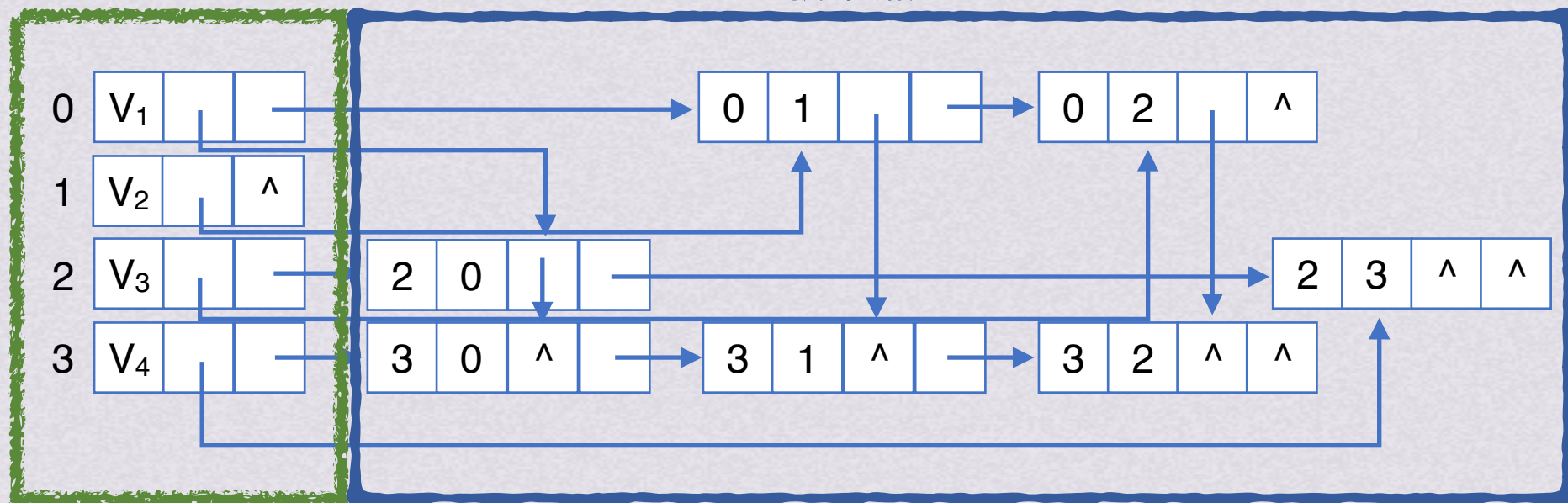
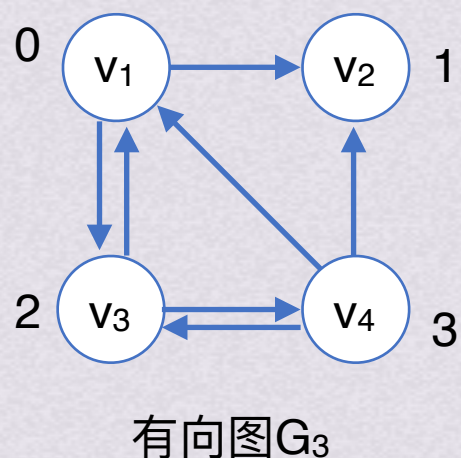


弧节点

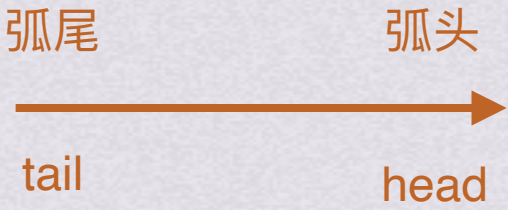


顶点节点

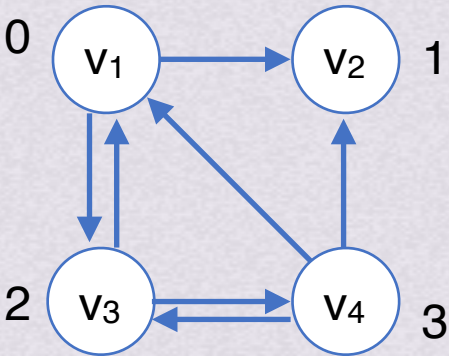
弧节点



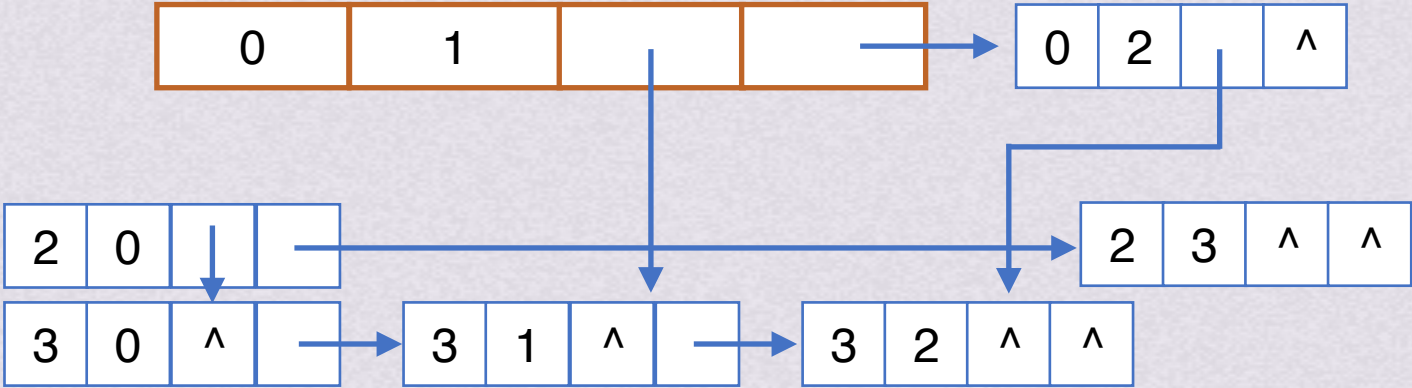
图的存储结构十字链表



弧节点

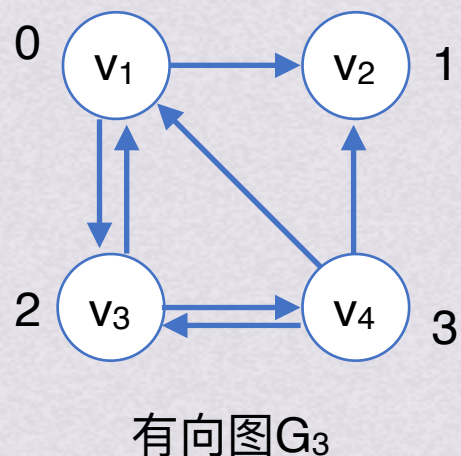
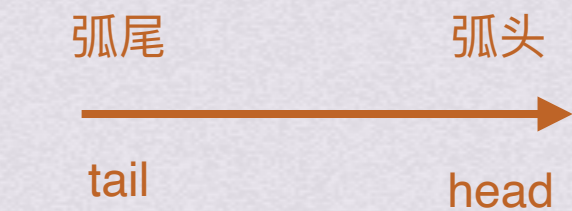


有向图G₃

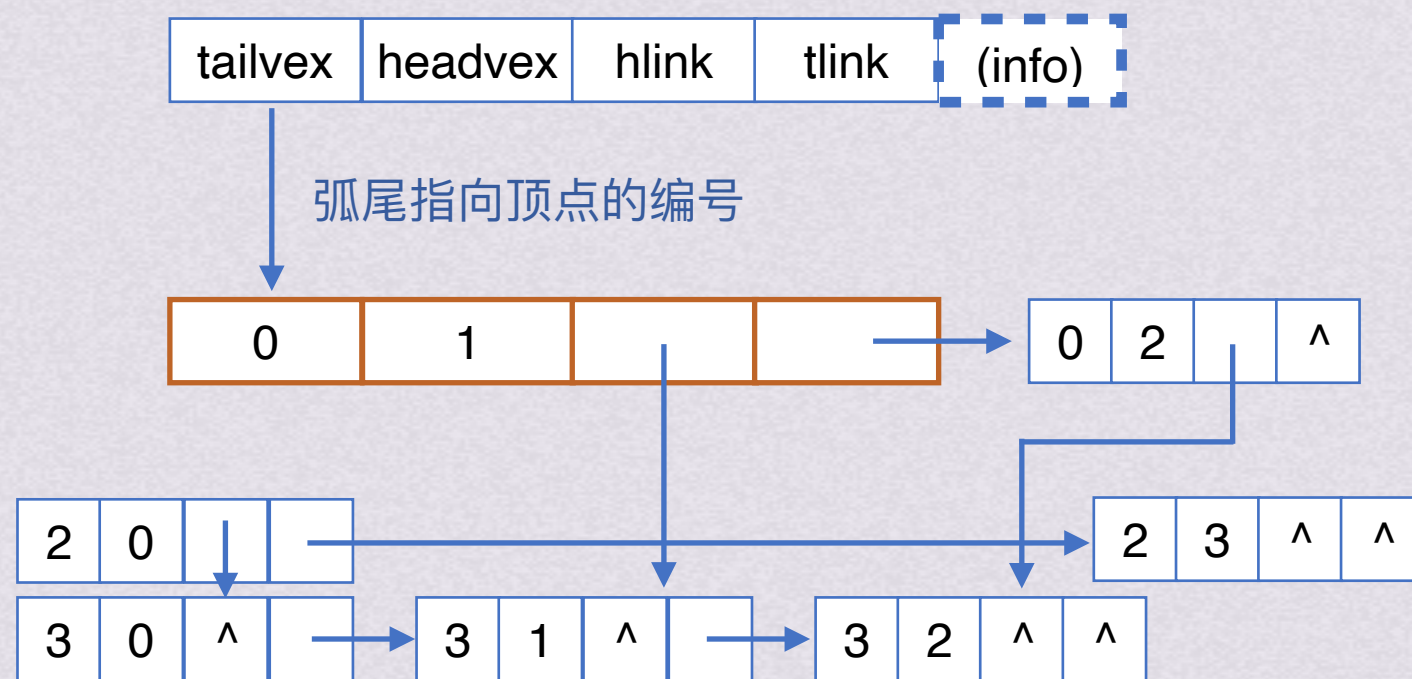




图的存储结构十字链表

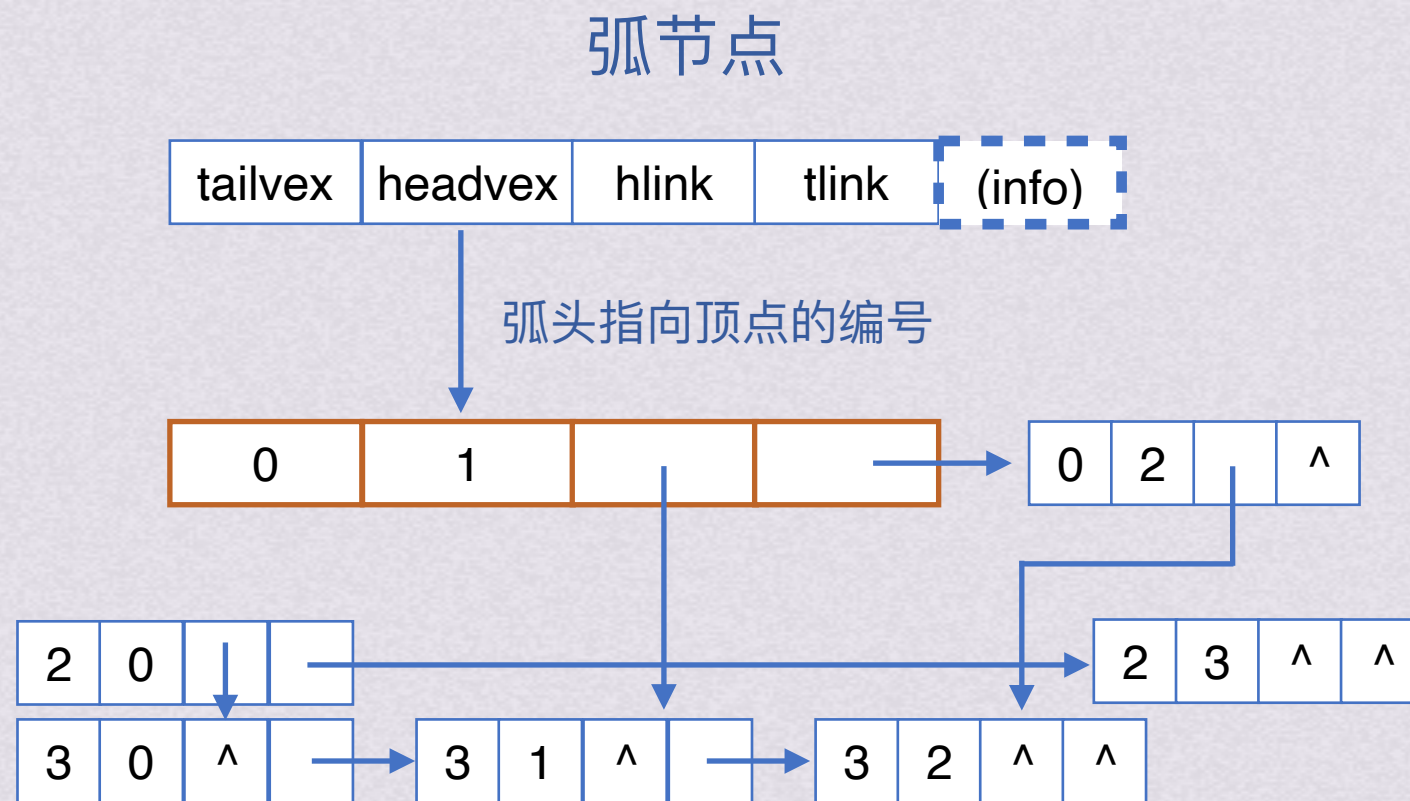
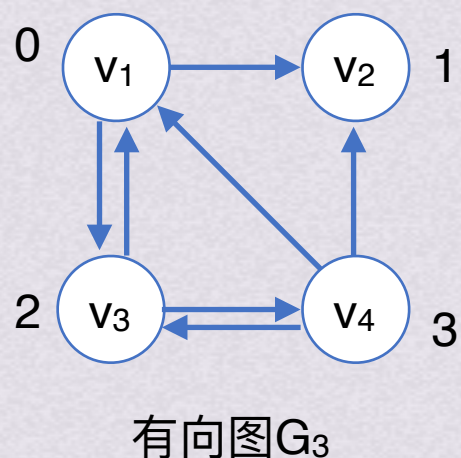
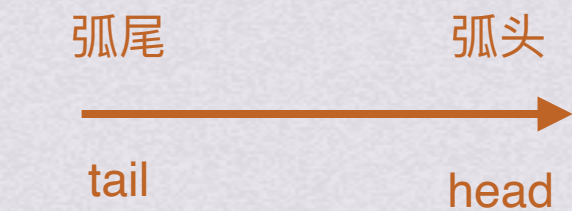


弧节点



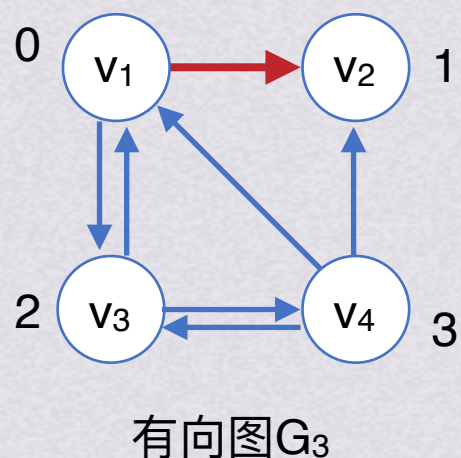
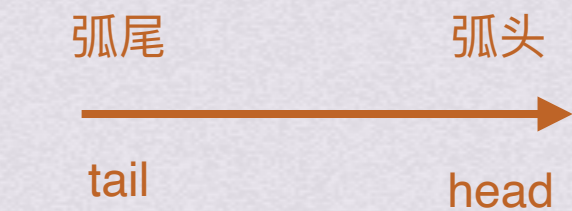


图的存储结构十字链表

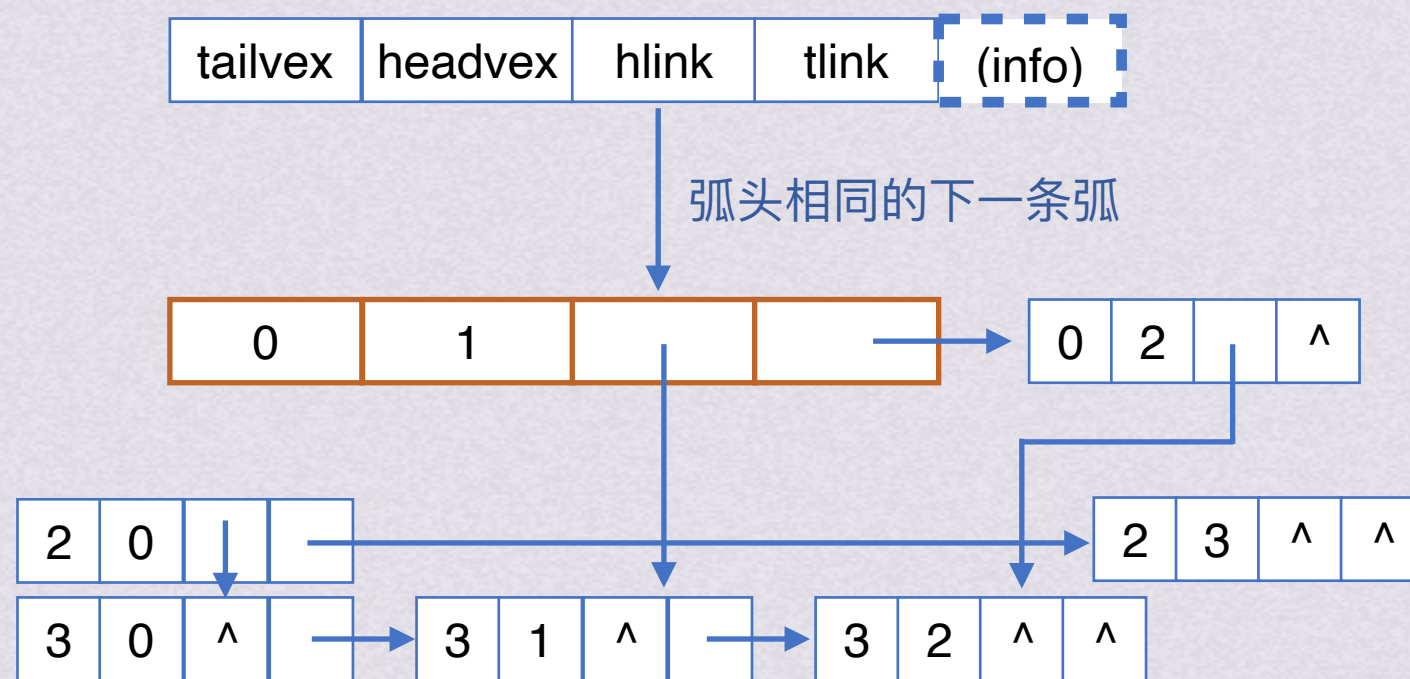




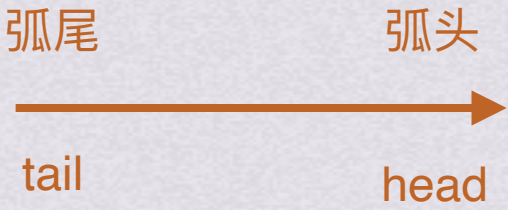
图的存储结构十字链表



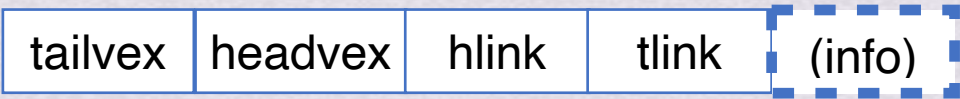
弧节点



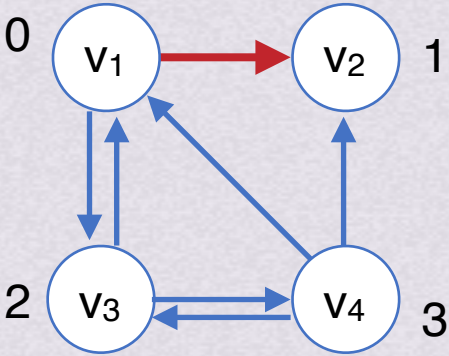
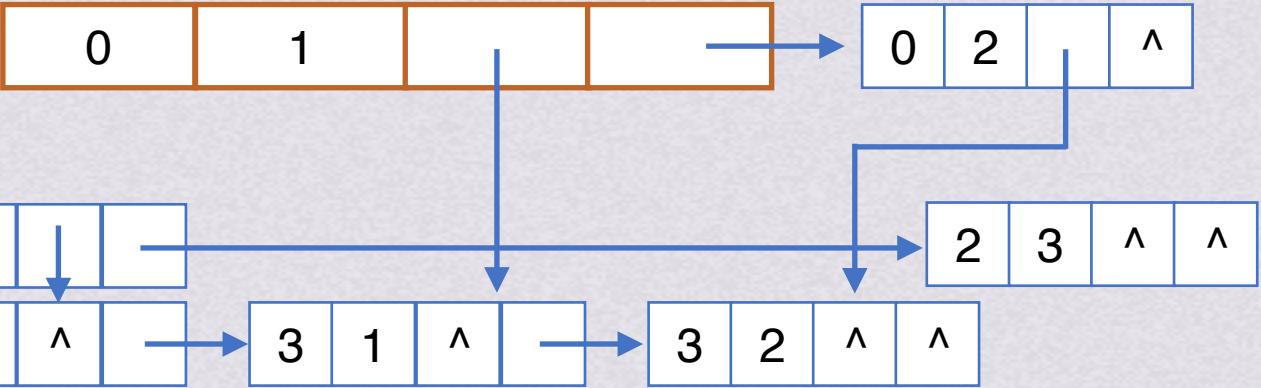
图的存储结构十字链表



弧节点

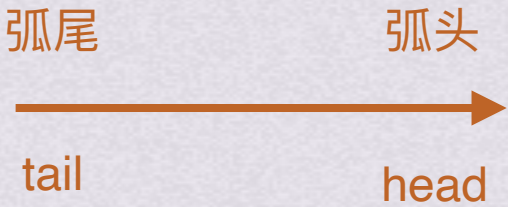


弧尾相同的下一条弧



有向图G₃

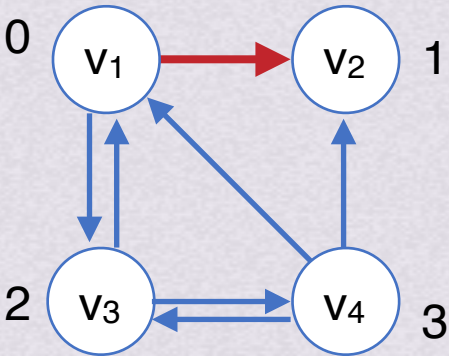
图的存储结构十字链表



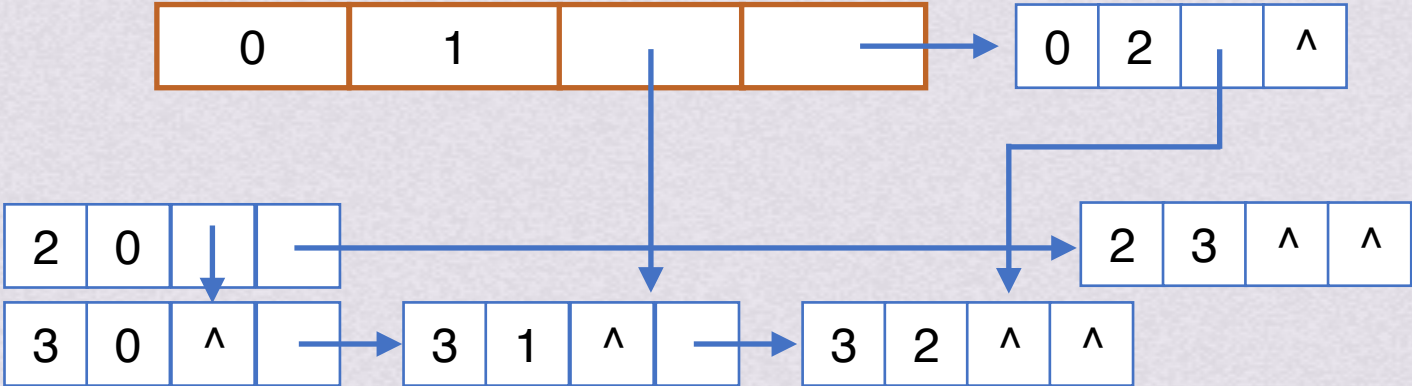
弧节点



弧的信息，
一般画图不画



有向图G₃





图的存储结构十字链表

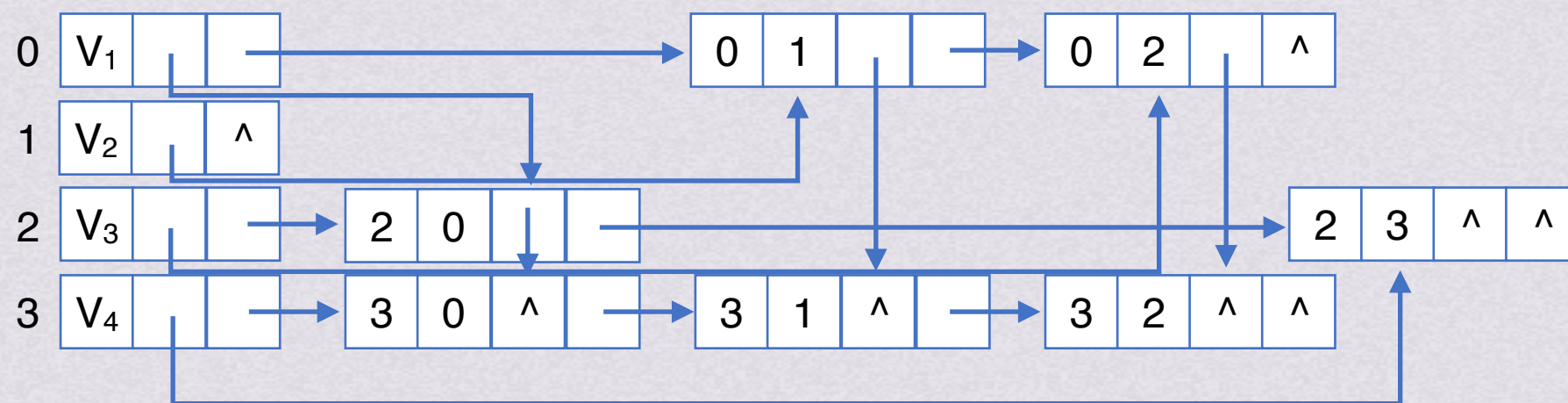
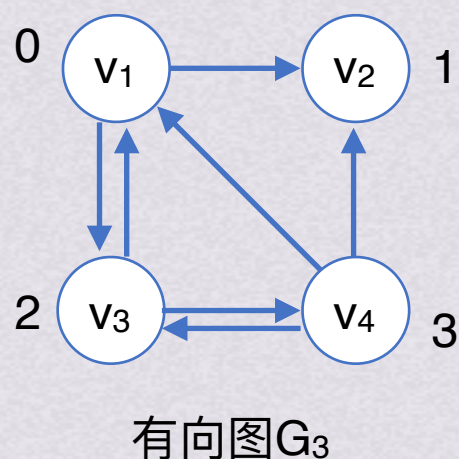
顶点节点

节点编号

data	firstin	firstout
------	---------	----------

弧节点

tailvex	headvex	hlink	tlink	(info)
---------	---------	-------	-------	--------



试试重新把十字链表画出来!



十字链表 我直接一个突然插入！

 4-2数组：稀疏矩阵

矩阵中非零元素的个数远远小于矩阵元素的个数

$$\begin{bmatrix} 3 & 0 & 0 & 5 \\ 0 & -1 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{bmatrix}$$

如何存储？

十字链表

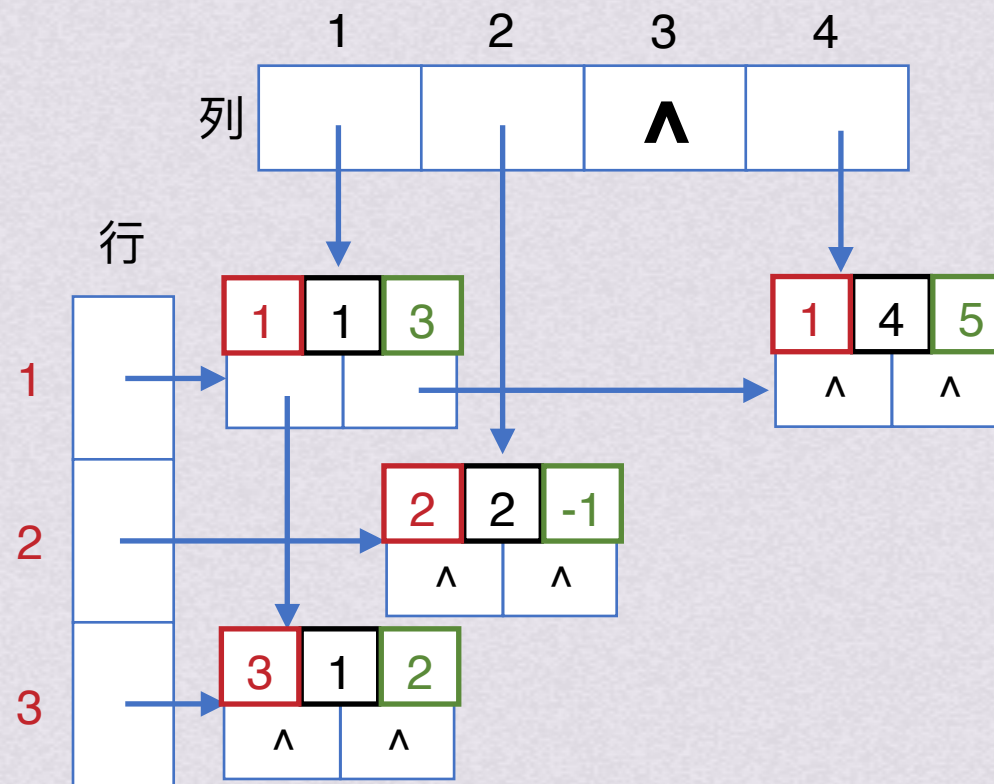
三元组



4-2数组：稀疏矩阵十字链表

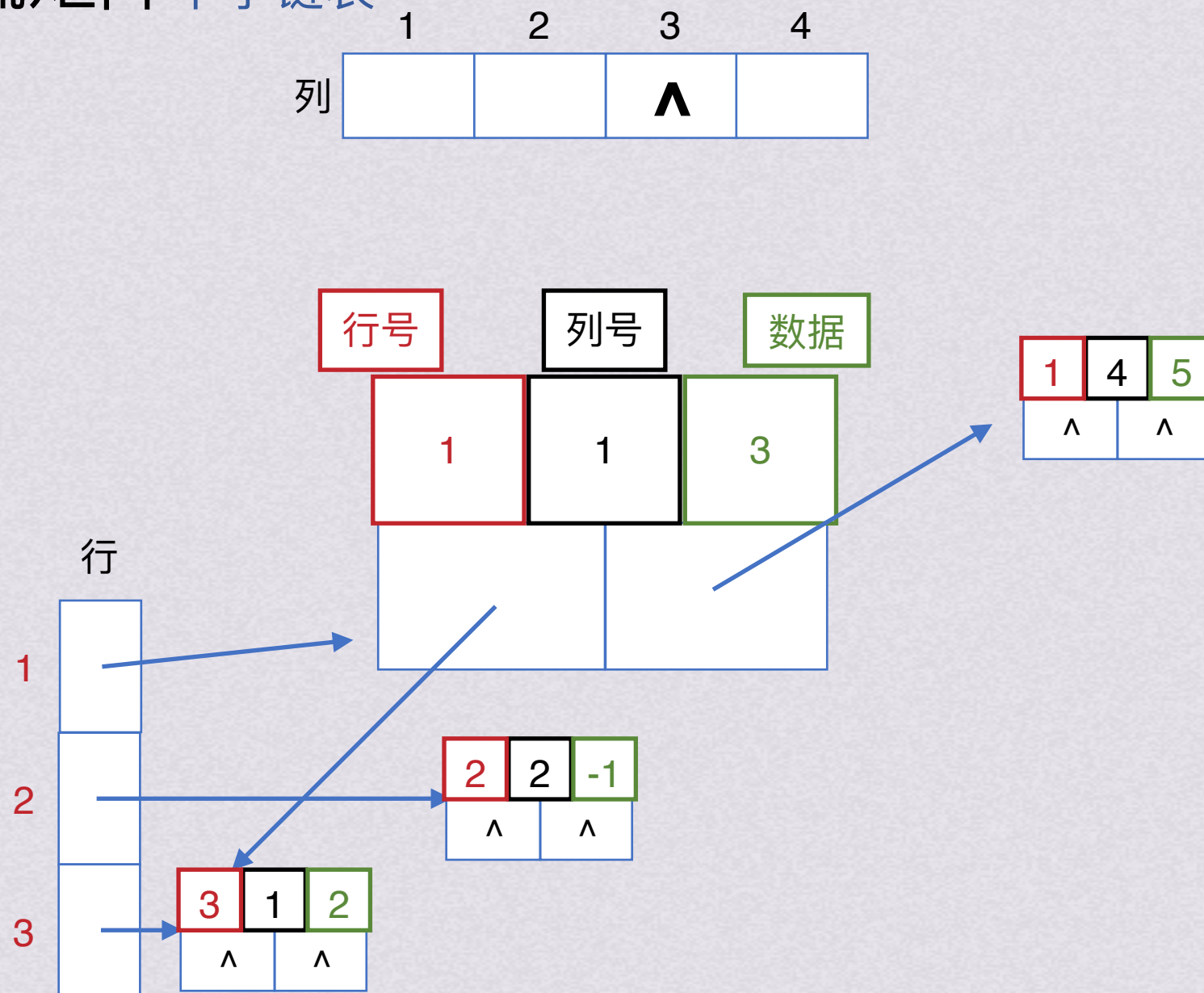
十字链表存储矩阵

	3	0	0	5
	0	-1	0	0
	2	0	0	0





4-2数组：稀疏矩阵十字链表





4-2数组：稀疏矩阵三元组

三元组存储矩阵

	3	0	0	5
	0	-1	0	0
	2	0	0	0

i	j	data
0	0	3
0	3	5
1	1	-1
2	0	2

对应三元组表



图的存储结构

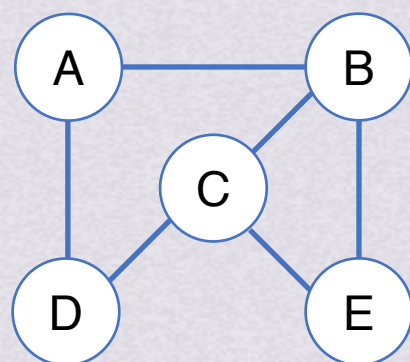
邻接多重表 (24刚考过!)
(无向图)

顶点节点

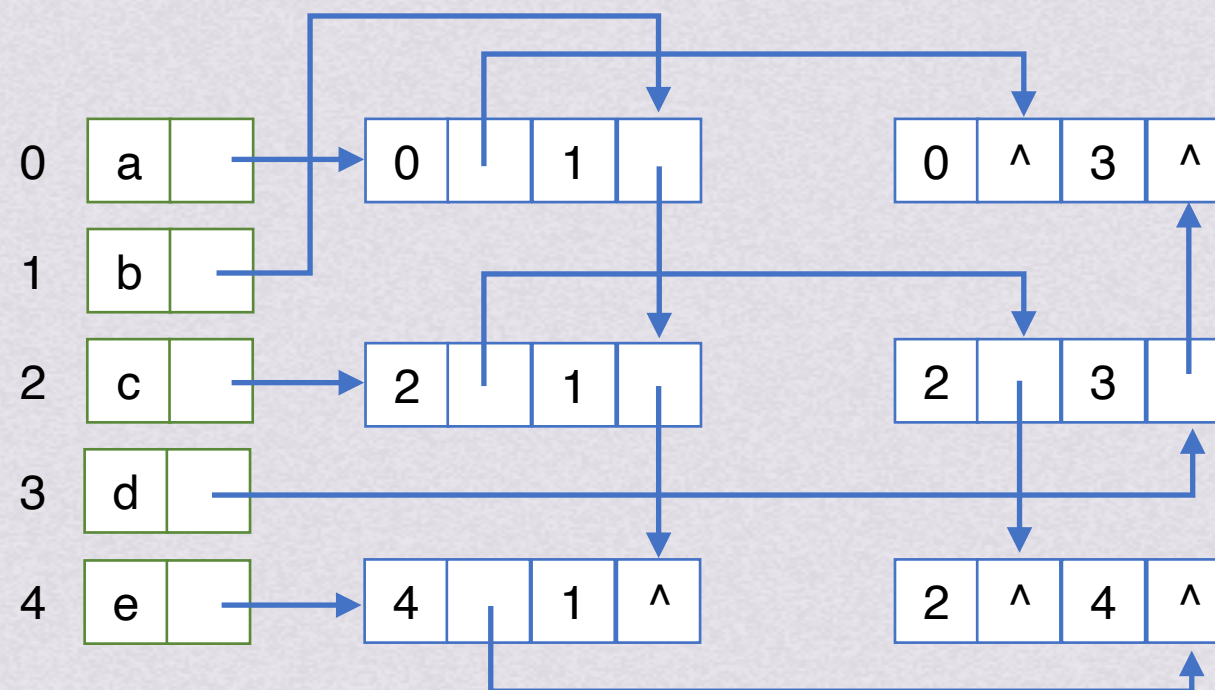
data	firstedge
------	-----------

弧节点

ivex	ilink	jvex	jlink	info
------	-------	------	-------	------



无向图 G_2





图的存储结构

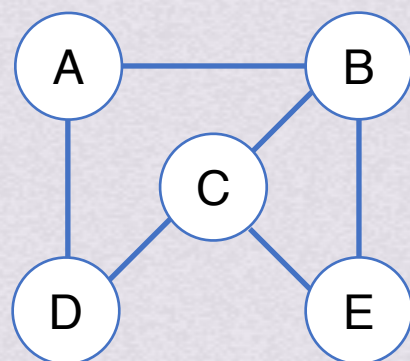
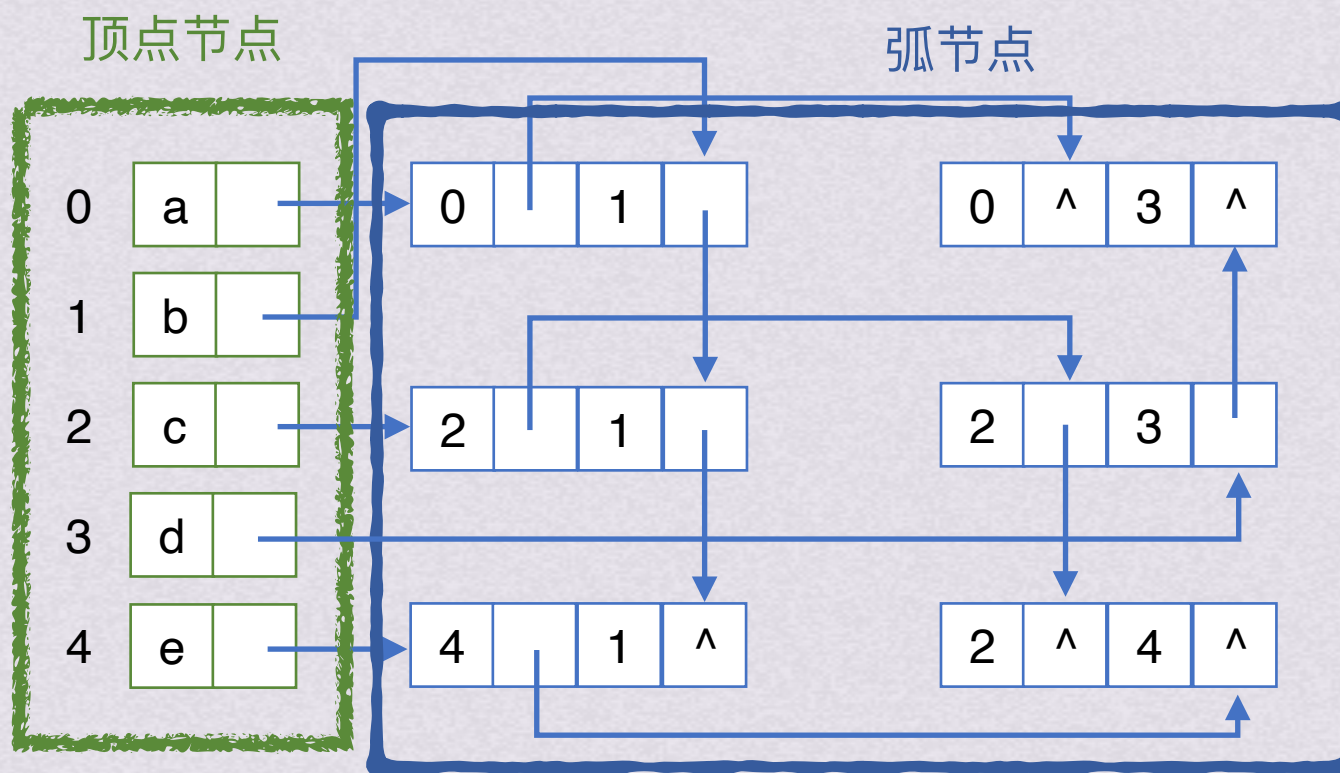
邻接多重表 (24刚考过!)
(无向图)

顶点节点

data	firstedge
------	-----------

弧节点

ivex	ilink	jvex	jlink	info
------	-------	------	-------	------

无向图 G_2 



图的存储结构

邻接多重表 (24刚考过!)
(无向图)

弧节点

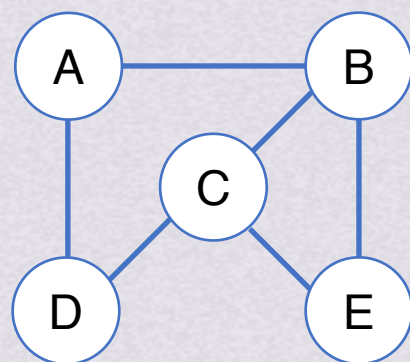
ivex

ilink

jvex

jlink

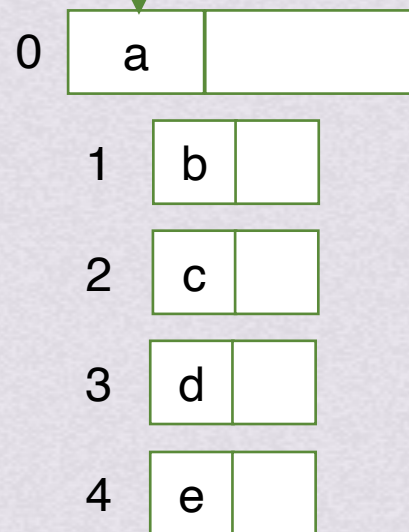
info

无向图 G_2

顶点节点



存储数据





图的存储结构

邻接多重表 (24刚考过!)
(无向图)

弧节点

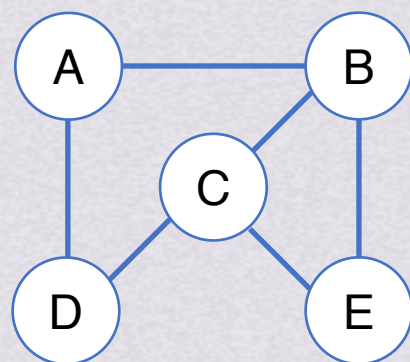
ivex

ilink

jvex

jlink

info

无向图 G_2

顶点节点

data

firstedge

指向第一条依附于该顶点的边

0

a

1

b

2

c

3

d

4

e



图的存储结构

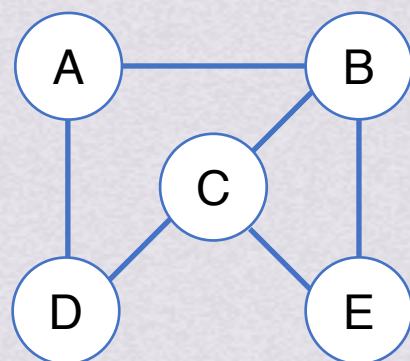
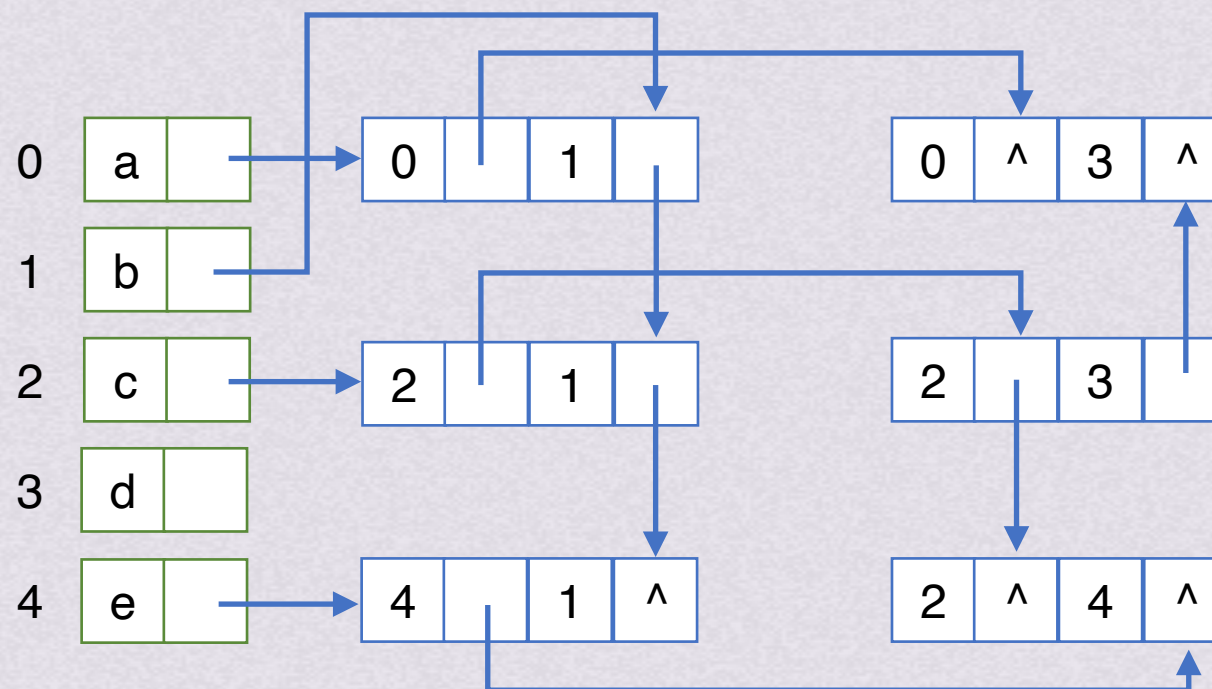
邻接多重表 (24刚考过!)
(无向图)

顶点节点

data	firstedge
------	-----------

弧节点

ivex	ilink	jvex	jlink	info
------	-------	------	-------	------

无向图 G_2 



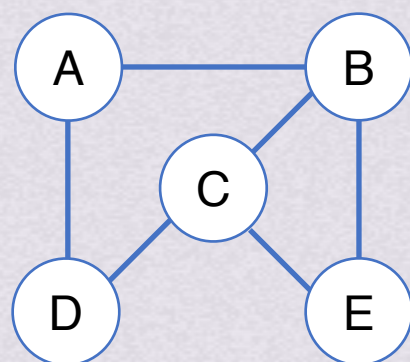
图的存储结构

邻接多重表 (24刚考过!)
(无向图)

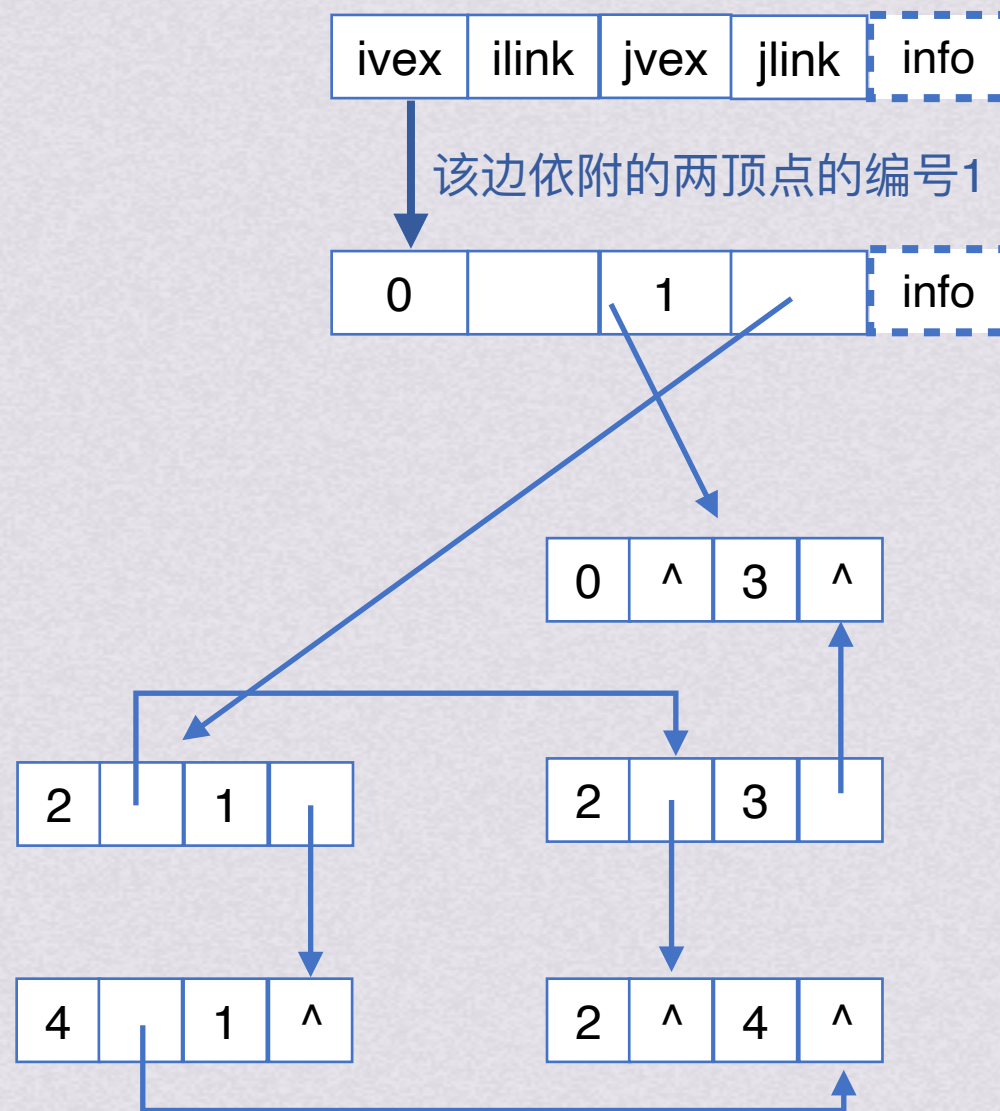
顶点节点

data	firstedge
------	-----------

弧节点



无向图G₂





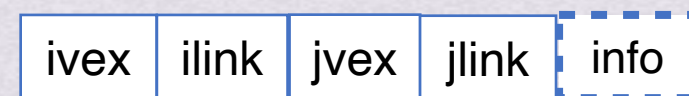
图的存储结构

邻接多重表 (24刚考过!)
(无向图)

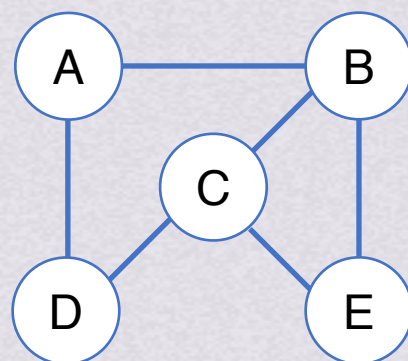
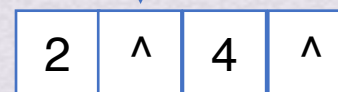
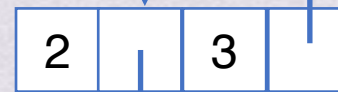
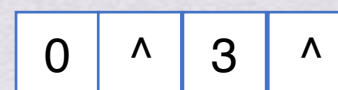
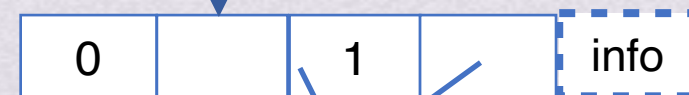
顶点节点

data	firstedge
------	-----------

弧节点



指向下一条依附于ivex的边



无向图G₂



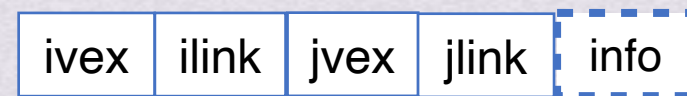
图的存储结构

邻接多重表 (24刚考过!)
(无向图)

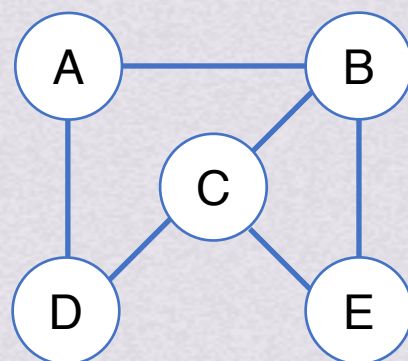
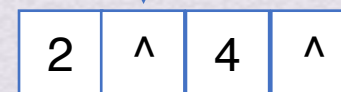
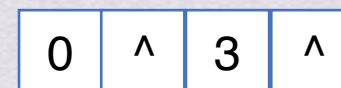
顶点节点

data	firstedge
------	-----------

弧节点



该边依附的两顶点的编号2



无向图G₂



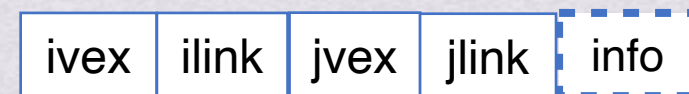
图的存储结构

邻接多重表 (24刚考过!)
(无向图)

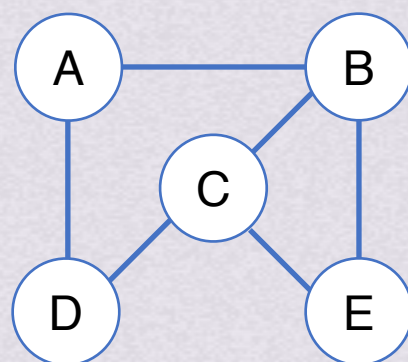
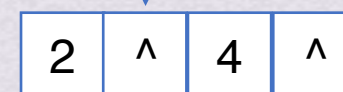
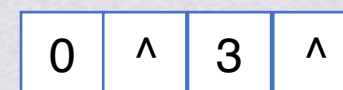
顶点节点

data	firstedge
------	-----------

弧节点



指向下一条依附于jvex的边



无向图G₂



图的存储结构

邻接多重表 (24刚考过!)
(无向图)

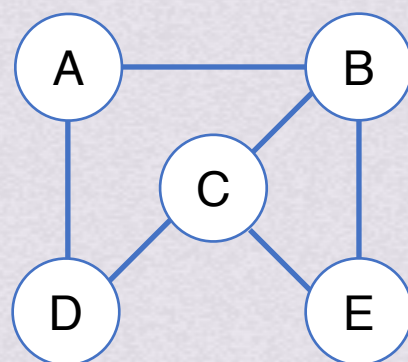
顶点节点

data	firstedge
------	-----------

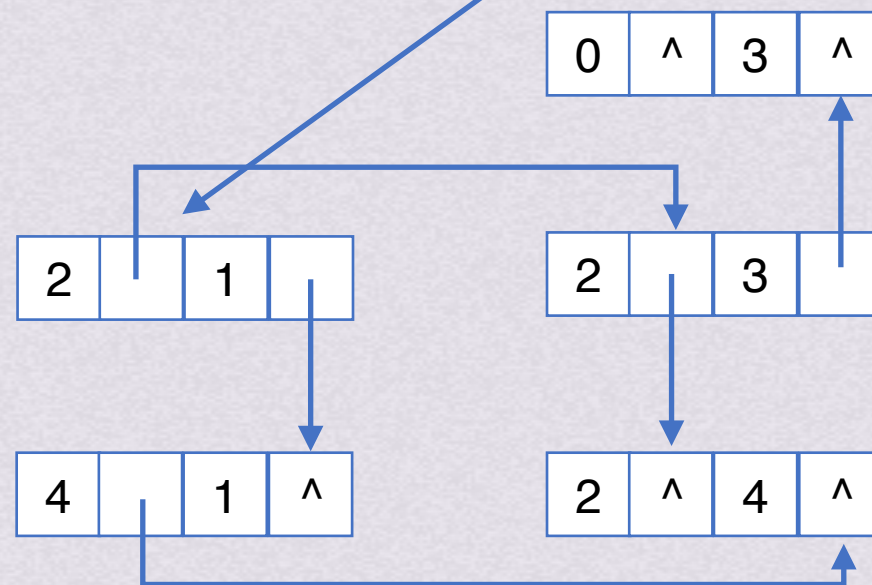
弧节点



存储边的信息
(画图一般不画出)



无向图G₂





图的存储结构

邻接多重表 (24刚考过!)

(无向图)

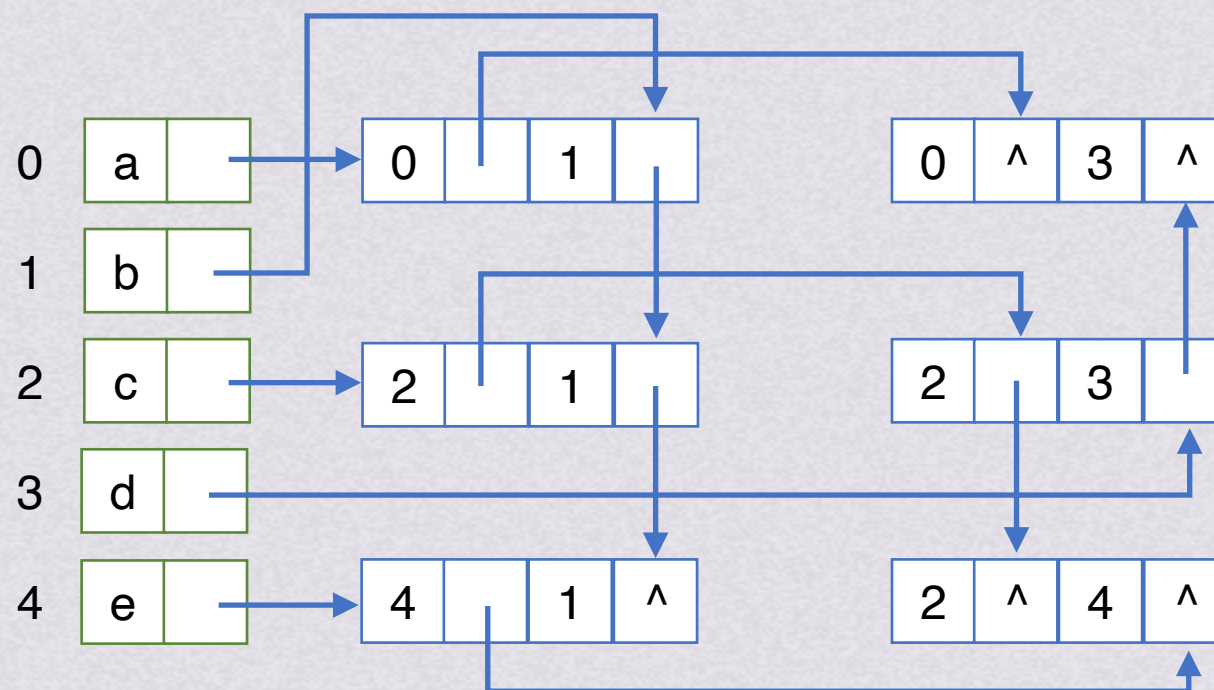
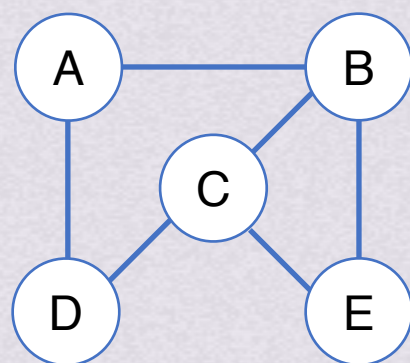
顶点节点

节点编号

data	firstedge
------	-----------

弧节点

ivex	ilink	jvex	jlink	info
------	-------	------	-------	------



试试重新把邻接多重表画出来!



图的存储结构

邻接多重表 (24刚考过!)

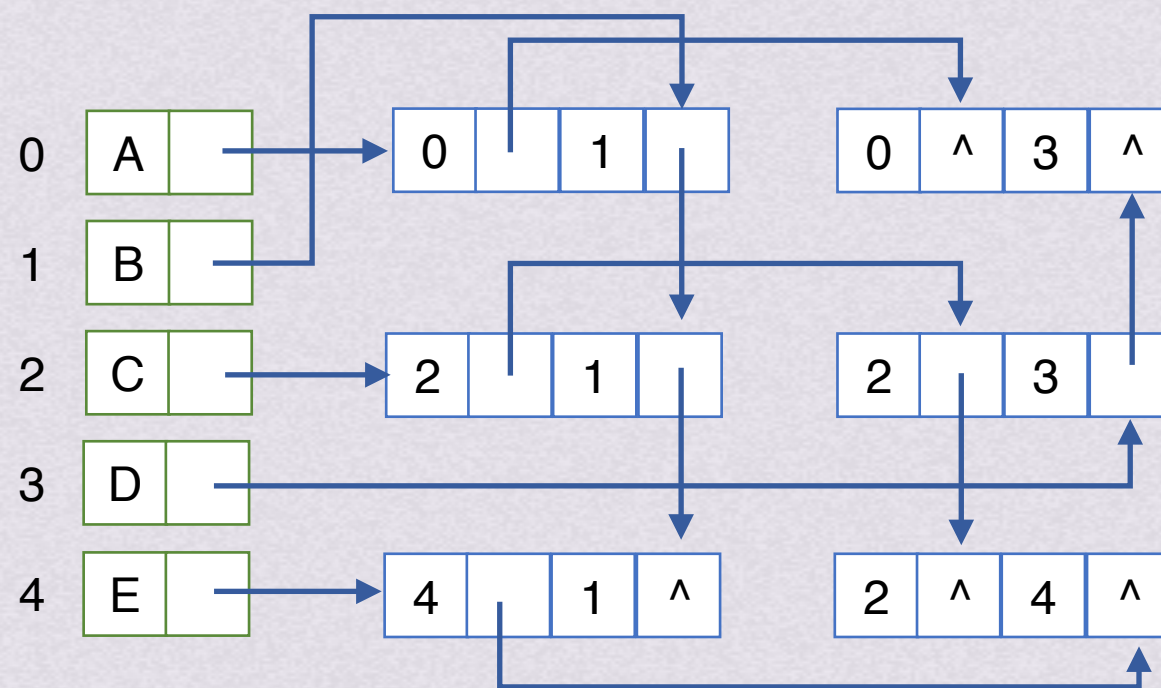
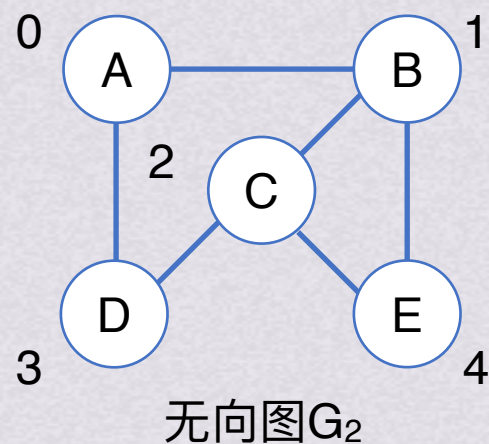
(无向图)

顶点节点 节点编号

data	firstedge
------	-----------

弧节点

ivex	ilink	jvex	jlink	info
------	-------	------	-------	------



*注：邻接表、十字链表、邻接多重表均不唯一
邻接矩阵唯一

比较项目		邻接矩阵		邻接表	
		无向图	有向图	无向图	有向图
空间		邻接矩阵对称，可压缩至 $n(n-1)/2$ 个单元	邻接矩阵不对称，存储 n^2 个单元	存储 $n+2e$ 个单元	存储 $n+e$ 个单元
时间	求某个顶点 V_i 的度	查找邻接矩阵中序号 i 对应的一行， $O(n)$	求出度：查找邻接矩阵的一行， $O(n)$ 求入度：查找矩阵的一列， $O(n)$	查找 V_i 的边表，最坏情况 $O(n)$	求出度：查找 V_i 的边表，最坏情况 $O(n)$ ； 求入度：按顶点表顺序查找所有边表， $O(n+e)$
	求边的数目	查找邻接矩阵， $O(n^2)$		按顶点表顺序查找所有边表， $O(n+2e)$	按顶点表顺序查找所有边表， $O(n+e)$
	判定边 (V_i, V_j) 是否存在	直接检查邻接矩阵 $A[i][j]$ 元素的值， $O(1)$		查找 V_i 的边表，最坏情况 $O(n)$	
适用情况		稠密图		稀疏图	